
操作系统课程设计报告

MIT 6.1810 xv6 Labs 2025

学院	计算机科学与技术
专业	软件工程
姓名	李柏言
学号	2452281
指导教师	王冬青
完成日期	2026 年 8 月 20 日
源码仓库	https://github.com/lbyy718/xv6-labs-2025

摘要

本课程设计基于 MIT 6.1810 2025 版 xv6，完成 Unix utilities、System calls、Page tables、Traps、Copy-on-write、Network driver、Locking、File system 和 Memory mapping 九个 Lab。项目从用户程序与系统调用接口出发，逐步深入页表、陷阱、物理页生命周期、设备中断、并发同步和持久化存储，并最终在 mmap Lab 中把系统调用、文件系统和缺页处理组合为完整的惰性文件映射机制。实现过程中采用“阅读官方说明—分析评分脚本—分阶段实现—专项测试—完整回归”的方法，以 Git 分支和小步提交保存各阶段结果。关键工作包括系统调用策略控制、超级页映射、COW 引用计数、网卡描述符环、每 CPU 空闲页链表、写者优先读写锁、二级间接块、符号链接以及 VMA 的创建、装页、写回和回收。九个 Lab 的官方评分均全部通过。实验表明，内核功能的正确性不仅取决于局部算法，还取决于权限、锁、引用计数、资源所有权及异常路径能否形成闭合生命周期。

关键词：xv6；RISC-V；操作系统；系统调用；虚拟内存；文件系统

目录

摘要	I
1. 项目概述	1
1.1 项目背景	1
1.2 项目目标	1
1.3 项目分工	1
1.4 完成情况总览	1
2. 开发环境、工具与调试方法	2
2.1 开发环境	2
2.2 环境安装与验证	2
2.3 仓库与分支管理	3
2.4 通用调试方法	3
2.5 环境问题与解决方法	3
3. Lab util: Unix utilities	4
3.1 实验目的与要求	4
3.2 sleep: 用户级延时程序	4
3.3 sixfive: 文本文件中的数字筛选	5
3.4 memdump: 按格式解释连续内存	6
3.5 find: 递归查找目录树	8
3.6 find -exec: 对匹配文件执行命令	9
3.7 实验中遇到的问题和解决方法	10
3.8 最终验收	10
3.9 本 Lab 心得	11
4. Lab syscall: System calls	12
4.1 实验目的与要求	12
4.2 Using gdb: 跟踪系统调用与内核页错误	12
4.3 Sandbox a command: 按掩码限制系统调用	13
4.4 Sandbox with allowed pathnames: 精确路径例外	14
4.5 Attack xv6: 利用未清零物理页	15
4.6 实验中遇到的问题和解决方法	16
4.7 最终验收与心得	16
5. Lab pgtbl: Page tables	18
5.1 实验目的与要求	18
5.2 Inspect a user-process page table: 解释用户页表	18

5.3 Speed up system calls: 共享只读 USYSCALL 页	19
5.4 Print a page table: 递归打印三级映射	20
5.5 Use superpages: 2 MiB 大页映射	21
5.6 实验中遇到的问题和解决方法	22
5.7 最终验收与心得	23
6. Lab traps: Traps	25
6.1 实验目的与要求	25
6.2 RISC-V assembly: 调用约定与反汇编分析	25
6.3 Backtrace: 遍历内核栈帧	26
6.4 Alarm: 由时钟中断进入用户处理函数	27
6.5 实验中遇到的问题和解决方法	28
6.6 最终验收与心得	28
7. Lab cow: Copy-on-write	30
7.1 实验目的与要求	30
7.2 COW fork: 共享父进程物理页	30
7.3 写缺页: 按需创建私有副本	31
7.4 copyout: 处理内核发起的用户内存写入	31
7.5 物理页引用计数与并发	31
7.6 实验中遇到的问题和解决方法	32
7.7 最终验收与心得	32
8. Lab net: Network driver	34
8.1 实验目的与要求	34
8.2 Part One: E1000 NIC 驱动	34
8.3 Part Two: UDP Receive	36
8.4 实验中遇到的问题和解决方法	37
8.5 最终验收与心得	38
9. Lab lock: Locking	40
9.1 实验目的与要求	40
9.2 Memory allocator: 每 CPU 空闲链表	40
9.3 Read-write lock: 写者优先	42
9.4 实验中遇到的问题和解决方法	43
9.5 最终验收与心得	44
10. Lab fs: File system	46
10.1 实验目的与要求	46
10.2 Large files: 二级间接块	46

10.3 Symbolic links: 符号链接	48
10.4 实验中遇到的问题和解决方法	49
10.5 最终验收与心得	49
11. Lab mmap: Memory mapping	51
11.1 实验目的与要求	51
11.2 系统调用与 VMA: 只登记、不装页	51
11.3 页错误: 按文件偏移装入物理页	52
11.4 <code>mmap</code> : 部分解除与共享写回	53
11.5 <code>fork</code> 、 <code>exit</code> 与资源生命周期	53
11.6 实验中遇到的问题和解决方法	53
11.7 最终验收与心得	54
12. 综合分析	56
12.1 系统调用的完整链路	56
12.2 进程、陷阱与虚拟内存的关系	56
12.3 并发、锁与资源生命周期	56
12.4 各 Lab 之间的联系	57
13. 总结与心得	58
14. 参考资料	59
15. 源码仓库与提交记录	60
附录: 完整评分结果	61

1. 项目概述

1.1 项目背景

xv6 是一个面向教学的 Unix 风格操作系统。本项目以 MIT 6.1810 2025 版 xv6 Labs 为基础，通过阅读内核源码、完成实验任务和分析运行结果，理解进程、系统调用、页表、中断与异常、并发控制及文件系统等核心机制。

1.2 项目目标

- 掌握 xv6 的编译、启动、测试和调试流程。
- 理解用户程序到内核服务的完整调用链。
- 理解虚拟内存、陷阱、网络、锁和文件系统的核心实现。
- 通过手工测试、局部评分和完整评分验证实现正确性。

1.3 项目分工

本项目由本人独立完成，负责九个 xv6 Lab 的源码实现、测试验证、Git 分支管理和 最终报告整理。各实验按官方同名分支开发，报告统一在 `report` 分支维护。

1.4 完成情况总览

Lab	主题	实现状态	make grade	报告状态
util	Unix utilities	已完成	131/131	已完成
syscall	System calls	已完成	45/45	已完成
pgtbl	Page tables	已完成	41/41	已完成
traps	Traps	已完成	95/95	已完成
cow	Copy-on-write	已完成	130/130	已完成
net	Network driver	已完成	171/171	已完成
lock	Locking	已完成	100/100	已完成
fs	File system	已完成	100/100	已完成
mmap	Memory mapping	已完成	170/170	已完成

2. 开发环境、工具与调试方法

2.1 开发环境

组件	实际版本	用途
Windows + WSL 2	WSL2 内核 6.18.33.2-microsoft-standard	宿主和 Linux 开发环境
Ubuntu	24.04	执行 Git、Make 和评分脚本
RISC-V GCC	13.3.0	交叉编译 xv6
QEMU	8.2.2	模拟 RISC-V 硬件
VS Code (WSL)	1.134.0	源码阅读和编辑

2.2 环境安装与验证

本项目采用 Windows 主机与 WSL 2 中 Ubuntu 24.04 的组合环境。首次搭建时，先在具有管理员权限的 Windows PowerShell 中安装 Ubuntu，并确认发行版运行在 WSL 2 模式：

```
wsl --install -d Ubuntu-24.04
wsl --set-default-version 2
wsl --list --verbose
```

首次进入 Ubuntu 时创建 Linux 用户并设置密码。随后更新软件索引，安装 xv6 编译、运行、调试和评分所需的工具：

```
sudo apt update
sudo apt install -y git build-essential python3 gdb-multiarch \
  qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

源码通过 GitHub 托管仓库取得，并保留 MIT 官方仓库作为基线远程。这样既能在个人远程保存各 Lab 的实现，也能随时对照官方分支：

```
cd ~
git clone https://github.com/lbyy718/xv6-labs-2025.git
cd xv6-labs-2025
git remote add mit git://g.csail.mit.edu/xv6-labs-2025
git fetch mit
git branch --all
```

如果仓库中已经存在名为 `mit` 的远程，则不需要重复执行 `git remote add`。本项目使用安装在 Windows 侧的 VS Code，通过 WSL 扩展打开 Linux 目录；在仓库根目录执行 `code .` 即可进入开发环境，终端、编译器和 Git 均运行在 Ubuntu 中。

安装完成后依次检查系统、交叉编译器和模拟器版本，并编译启动 xv6：

```
uname -r
lsb_release -ds
```

```
qemu-system-riscv64 --version
riscv64-linux-gnu-gcc --version
cd ~/xv6-labs-2025
make qemu
```

实际验证结果：`qemu-system-riscv64 --version` 返回 QEMU 8.2.2，`riscv64-linux-gnu-gcc --version` 返回 GCC 13.3.0；在仓库根目录执行 `make qemu` 可以启动 xv6 并进入用户 shell。完成测试后依次按 `Ctrl+A` 和 `x` 退出 QEMU。各 Lab 的完整评分结果见第 1.4 节和附录。

2.3 仓库与分支管理

项目使用双远程结构：`mit` 保留 MIT 官方实验分支，`origin` 保存个人实现和报告。各 Lab 在对应分支开发，以便将实现与官方基线进行比较。

2.4 通用调试方法

1. 先阅读第一条编译错误，再处理后续连锁报错。
2. 使用 `printf` 和最小测试用例缩小问题范围。
3. 使用 `addr2line`、`kernel.asm` 和 GDB 分析 panic 或卡死。
4. 先运行局部测试，最后从干净编译状态运行 `make grade`。

2.5 环境问题与解决方法

现象	定位方法	根本原因	解决方法	验证结果
<code>rg</code> 命令不可用	执行 Backtrace 地址提取命令时报错	Ubuntu 环境未安装 <code>ripgrep</code>	改用 <code>grep -oE</code> 和 <code>while read</code> 管道完成地址解析	<code>addr2line</code> 解析正常
将 xv6 内命令直接交给 Bash	把 <code>test basic mmap</code> 当作宿主命令执行	xv6 测试命令必须通过评分脚本或 QEMU shell 运行	使用 <code>./grade-lab-mmap mmaptest</code> 或在 QEMU 中运行 <code>mmaptest</code>	<code>mmap</code> 测试通过
多分支合并出现冲突	报告和实验分支包含不同文件历史	分支职责不同，直接合并会产生同名文件冲突	实验实现保留在各 Lab 分支，报告集中提交到 <code>report</code>	分支和远程状态一致

3. Lab util: Unix utilities

3.1 实验目的与要求

项目	内容
官方分支	util
实验主题	xv6 用户程序、文件 I/O、C 内存表示、目录遍历和进程控制
具体任务	sleep、sixfive、mempdump、find、find -exec
基线提交	db9a9d8
完成提交	5493306
实际用时	6 小时
最终评分	131/131

本 Lab 的任务集合与旧版 xv6 Lab 不同。开发前先对照 2025 官方页面、本地 grade-lab-util 和当前分支文件，确定了真实任务及评分边界，再按一个功能一个 Git 检查点的方式实现。

3.2 sleep: 用户级延时程序

3.2.1 实验目的

实现用户级 sleep 命令，将命令行中的 ticks 参数转换为整数，并调用 xv6 2025 版已提供的 pause() 系统调用暂停当前进程。

3.2.2 原理与调用链

shell 解析 sleep 10
→ main(argc, argv)
→ atoi(argv[1])
→ pause(ticks) 用户态桩
→ sys_pause()
→ 根据时钟中断更新的 ticks 休眠与唤醒

2025 版的内核入口是 sys_pause，不是旧版报告常见的 sys_sleep。kernel/sysproc.c、user/user.h 和系统调用桩均已存在，因此本任务只需增加用户程序，无需修改内核。

3.2.3 设计与实现

文件	修改	作用
user/sleep.c	新建用户程序	验证参数、转换 ticks、调用 <code>pause()</code>
Makefile	在 <code>UPROGS</code> 中加入 <code>_sleep</code>	将程序编译并写入 xv6 文件系统镜像

```

if(argc != 2){
    fprintf(2, "usage: sleep ticks\n");
    exit(1);
}
pause(atoi(argv[1]));
exit(0);

```

参数数量必须精确为 1，避免读取不存在或多余的参数。无参数运行时向标准错误输出用法，并以非零状态退出。

3.2.4 实验结果与分析

测试	结果
无参数运行	输出 <code>usage: sleep ticks</code> 并正常返回 shell
<code>sleep 10</code>	暂停约 10 ticks 后返回 shell
<code>./grade-lab-util sleep</code>	3/3 测试通过，包括断点验证 <code>sys_pause</code> 被调用

评分脚本不仅检查程序是否返回，还在 `sys_pause` 设置断点，因此全部通过说明程序确实进入了要求的系统调用，而不是用空循环模拟延时。

3.3 sixfive: 文本文件中的数字筛选

3.3.1 实验目的

使用 `open()` 和 `read()` 逐字符读取一个或多个文件，识别由官方指定分隔符包围的十进制数字，输出能被 5 或 6 整除的值。

3.3.2 解析设计

实现使用三个状态量：

- `valid`：当前 token 是否仍由纯数字构成；
- `have_digits`：当前 token 是否至少读到一个数字；
- `value`：按十进制累积的数值。

遇到空格、减号、回车、制表符、换行、点、斜杠、逗号或 EOF 时结算 token。如果 token 中出现其他非数字字符，则整个 token 无效，直到下一分隔符才重置。这保证了 06 会作为数值 6 输出，而 xv6 中的字符 6 不会被误识别。

3.3.3 实现与结果

文件	修改	作用
user/sixfive.c	新建逐字符解析程序	处理 token、整除判断、多文件和错误路径
Makefile	在 UPROGS 中加入 _sixfive	将程序加入镜像

```
$ sixfive sixfive.txt README
5
100
18
6
6
6
1810
6
1810
```

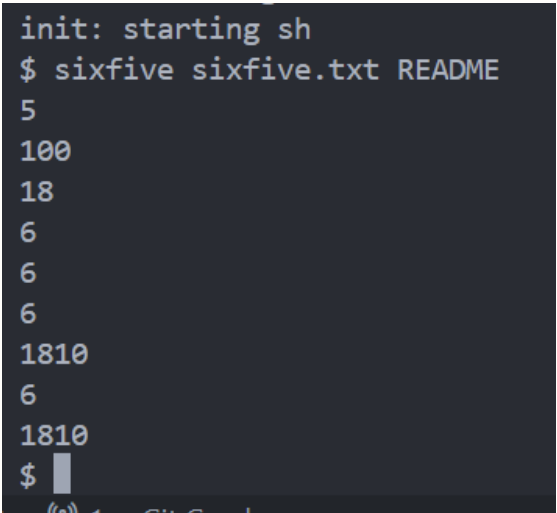


图 3-1 sixfive 对 sixfive.txt 和 README 的多文件处理结果

./grade-lab-util sixfive 中的 sixfive_test、sixfive_readme 和 sixfive_all 均通过，分别验证了基本数字解析、文本内嵌数字边界和多文件顺序处理。

3.4 memdump：按格式解释连续内存

3.4.1 实验目的

实现 memdump(char *fmt, char *data)，理解 C 语言中整数、字符、指针、结构体对齐和字符串在连续内存中的表示。

3.4.2 格式字符与指针推进

格式	解释方式	data 推进
i	32 位有符号整数, 十进制	4 字节
p	64 位数值, 十六进制	8 字节
h	16 位有符号整数, 十进制	2 字节
c	8 位 ASCII 字符	1 字节
s	当前 8 字节保存的字符串指针	8 字节
S	当前位置的 NUL 结尾内联字符串	字符串长度 + 1

多字节字段没有直接将 `char *` 强制转换后解引用, 而是先使用 `memmove` 复制到正确类型的局部变量。这样既使字节宽度明确, 也避免了未对齐地址的直接访问。

3.4.3 实验结果与分析

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ memdump
Example 1:
61810
2025
Example 2:
a string
Example 3:
another
Example 4:
BA8
1819438967
100
z
xyzy
Example 5:
hello
w
o
r
l
d
$
```

图 3-2 memdump 对整数、指针、字符和字符串的解析结果

内置五组示例全部得到预期输出。图 3-2 中 Example 4 的首行是运行时指针值，具体地址每次可能不同，不影响正确性。./grade-lab-util memdump 的内置示例和标准输入组合两项测试均通过。

3.5 find：递归查找目录树

3.5.1 实验目的与调用链

实现简化版 Unix `find`，使用 xv6 的文件和目录系统调用递归查找指定名称的文件。

```
open(path)
→ fstat(fd, &st)
├─ T_FILE: 取路径最后一段，使用 strcmp() 匹配
└─ T_DIR: read(fd, &dirent, sizeof(dirent))
    → 构造子路径
    → 递归 find(child, name)
```

3.5.2 设计与实现

关键点	处理方式
目录项名称	复制固定长度 <code>DIRSIZ</code> 后显式补 <code>\0</code>
递归循环	跳过 inode 为 0 的目录项及 <code>.</code> 、 <code>..</code>
路径匹配	提取最后一段后使用 <code>strcmp()</code> ，不使用指针 <code>==</code>
路径安全	在 512 字节缓冲区中追加目录项前检查长度
资源管理	打开或 <code>fstat</code> 失败时报错，所有返回路径正确 <code>close(fd)</code>

`./grade-lab-util 'find, in' 'find, recursive'` 验证了当前目录、指定子目录和多层递归三种情况，全部通过。

3.6 find -exec: 对匹配文件执行命令

3.6.1 实验目的

在不破坏基础 `find path name` 行为的前提下，支持：

`find path name -exec command [args ...]`

对每个匹配文件，实际执行 “`command`、原命令参数、匹配路径”。

3.6.2 进程控制与参数构造

文件名匹配

- 复制 `-exec` 后的原参数
- 追加匹配文件路径
- `fork()`
 - ├ 子进程: `exec(argv[0], argv)`
 - └ 父进程: `wait(0)`

参数数组使用 `kernel/param.h` 中的 `MAXARG` 作为上限，并为追加的匹配路径和末尾空指针预留位置。父进程等待子进程结束，使多个匹配文件的命令输出保持可预期顺序。

3.6.3 回归测试与分析

```
$ $ QEMU: terminated
lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$ ./grade-lab-util 'find, in' 'find, recursive' exec
make: 'kernel/kernel' is up to date.
== Test find, in current directory == find, in current directory: OK (1.3s)
== Test find, in sub-directory == find, in sub-directory: OK (1.0s)
== Test find, recursive == find, recursive: OK (1.1s)
== Test exec == exec: OK (0.8s)
== Test exec, multiple args == exec, multiple args: OK (1.1s)
== Test exec, recursive find == exec, recursive find: OK (1.2s)
```

图 3-3 find 与 find -exec 的六项回归测试

图 3-3 同时运行了三项基础 `find` 和三项 `exec` 测试，全部为 `OK`。这说明执行扩展没有改变原有查找语义，同时支持单参数、多参数和递归目录中的命令执行。

3.7 实验中遇到的问题和解决方法

本 Lab 的问题主要来自版本差异、字节级数据处理和 `xv6` 的资源上限。处理时先用 官方 2025 页面与本地评分脚本确认要求，再用最小输入复现。

开始实现 `sleep` 时，旧版资料普遍调用 `sleep()`，而当前 2025 版要求使用 `pause()`，内核入口也改为 `sys_pause`。如果照搬旧版代码，程序虽然结构相似，却不符合本次评分点。最后以当前分支源码和评分脚本为准改用 `pause()`，断点测试 确认程序确实进入了 `sys_pause`。

实现 `sixfive` 时，最初只要遇到数字就开始累积，会把 `xv6` 中的字符 `6` 错当成独立数字。原因是字母虽然不是数字，却也不是官方规定的分隔符。后续增加 `valid` 状态，一旦 `token` 中出现非法字符，就丢弃整个 `token`，直到下一个合法分隔符才 重新开始。README 边界测试随后通过。

`memdump` 需要按 1 字节推进输入指针，但待读取的整数、短整数和指针可能位于未对齐 地址。直接把 `char *` 强制转换后解引用存在未对齐访问风险，因此改用 `memmove` 将固定宽度字节复制到对齐的局部变量，再按对应类型输出，所有格式测试均通过。

递归实现 `find` 时，`dirent.name` 是固定长度字段，不保证以 `\0` 结尾，直接交给 字符串函数会越界读取。解决方法是先复制 `DIRSIZ` 个字节并显式补结束符，再判断 `.`、`..` 和目标名称。扩展 `find -exec` 时还要为匹配路径及末尾空指针预留参数 位置，因此在构造参数前检查 `MAXARG`，避免数组越界。基础 `find` 和 `exec` 回归测试 均通过。

3.8 最终验收

完成所有任务后，在仓库根目录创建 `time.txt`，填写实际用时 6 小时。随后从干净编译状态运行：

```
make clean
make grade
```

验收项	结果
sleep	3 项通过
sixfive	3 项通过
memdump	2 项通过
find	3 项通过
find -exec	3 项通过
time	通过
最终总分	131/131

```
== Test sleep, no arguments ==
$ make qemu-gdb
sleep, no arguments: OK
== Test sleep, returns ==
$ make qemu-gdb
sleep, returns: OK (0.8s)
== Test sleep, makes syscall ==
$ make qemu-gdb
sleep, makes syscall: OK (0.9s)
== Test sixfive_test ==
$ make qemu-gdb
sixfive_test: OK (1.0s)
== Test sixfive_readme ==
$ make qemu-gdb
sixfive_readme: OK (1.2s)
== Test sixfive_all ==
$ make qemu-gdb
sixfive_all: OK (1.0s)
== Test memdump, examples ==
$ make qemu-gdb
memdump, examples: OK (0.7s)
== Test memdump, format ii, S, p ==
$ make qemu-gdb
memdump, format ii, S, p: OK (1.0s)
== Test find, in current directory ==
$ make qemu-gdb
find, in current directory: OK (1.1s)
== Test find, in sub-directory ==
$ make qemu-gdb
find, in sub-directory: OK (1.1s)
== Test find, recursive ==
$ make qemu-gdb
find, recursive: OK (1.0s)
== Test exec ==
$ make qemu-gdb
exec: OK (1.0s)
== Test exec, multiple args ==
$ make qemu-gdb
exec, multiple args: OK (0.9s)
== Test exec, recursive find ==
$ make qemu-gdb
exec, recursive find: OK (1.2s)
== Test time ==
time: OK
Score: 131/131
```

图 3-4 util Lab 完整评分结果

3.9 本 Lab 心得

本 Lab 的代码量不大，但每个任务都对应一类基础能力：`sleep` 连接了用户程序与系统调用，`sixfive` 训练了基于字节流的状态解析，`memdump` 直接展示了类型在内存中的字节表示，`find` 和 `find -exec` 则把目录结构、递归、文件描述符和进程控制串联起来。

实验过程中最重要的方法是先核对当前版本的任务和评分脚本，再将功能拆成小检查点。每一项都经过“编译—手工样例—局部评分—Git 提交”，最后再做整体回归，因此出现问题时能够明确它属于哪个功能。这种节奏也为后续更复杂的内核 Lab 建立了可复用的开发和验收流程。

4. Lab syscall: System calls

4.1 实验目的与要求

项目	内容
官方分支	syscall
实验主题	系统调用入口、进程级调用限制与内存隔离安全
具体任务	GDB 调试、系统调用掩码 sandbox、路径例外、内存残留攻击
基线提交	0e53502
完成提交	f031d96
实际用时	5 小时
最终评分	45/45

本 Lab 从用户态 `ecall` 的入口开始，先借助 GDB 观察陷阱现场，再新增 `interpose()` 系统调用，将调用限制保存为进程状态并继承给子进程。随后将 单纯的系统调用屏蔽扩展为带路径例外的策略，最后利用未清零物理页泄露另一 进程的秘密数据，从防御和攻击两个方向理解系统调用边界。

4.2 Using gdb: 跟踪系统调用与内核页错误

4.2.1 系统调用链

用户程序并不能直接调用内核 C 函数。`user/usys.S` 中的桩函数把系统调用号 写入 `a7` 并执行 `ecall`；处理器进入 supervisor mode 后由 trampoline 保存 寄存器，`usertrap()` 识别用户态环境调用，最后进入 `syscall()` 完成分派：

用户函数（如 `open`）

- `user/usys.S`: `a7 = SYS_open`, 执行 `ecall`
- `trampoline.S`: 保存用户寄存器
- `usertrap()`
- `syscall()`: 读取 `trapframe->a7`
- `syscalls[a7]()`
- `sys_open()`

在 `syscall()` 设置断点后，backtrace 显示其直接调用者为 `usertrap()`。首次命中断点时 `p->trapframe->a7 = 0xf`，对应 `SYS_open`，来自 `init` 打开 console 的操作。`sstatus = 0x200000022` 的 SPP 位为 0，说明陷阱发生前 CPU 运行在 user mode。

4.2.2 页错误定位

按实验要求临时执行 `num = *(int *)0` 后，内核报告 `scause=0xd`、`stval=0x0`，含义是读取地址 0 时发生 load page fault。根据 `sepc` 在 `kernel.asm` 中定位到 `lw a3, 0(zero)`，GDB 在同一地址断点也得到相同指令。故障进程为 PID 1 的 `init`。原因是 xv6 内核页表没有映射虚拟地址 0，而不是系统调用数组本身发生越界。

```
(gdb) b syscall
Breakpoint 1 at 0x80002142: file kernel/syscall.c
(gdb) c
Thread 2 hit Breakpoint 1, syscall () at kernel/syscall.c
(gdb) backtrace
#0  syscall () at kernel/syscall.c
#1  usertrap () at kernel/trap.c

(gdb) p/x p->trapframe->a7
$1 = 0xf
# 0xf is SYS_open; a7 carries the syscall number.
(gdb) p/x $sstatus
$2 = 0x200000022
# SPP=0: the previous privilege mode was user mode.

panic test: sepc=0x80001cf8 scause=0xd stval=0x0
0x80001cf8: lw a3, 0(zero)
process: init, pid: 1
```

图 4-1 GDB 中的系统调用调用栈、特权级与空地址页错误定位

调试完成后恢复 `kernel/syscall.c`，没有把故意制造 panic 的代码带入最终提交。六个问题的完整观察结果记录在 `answers-syscall.txt` 中。

4.3 Sandbox a command: 按掩码限制系统调用

4.3.1 接口与状态设计

新增系统调用接口为：

```
int interpose(int mask, const char *allowed_path);
```

`mask` 的第 `n` 位控制编号为 `n` 的系统调用。若该位为 1，调用默认被拒绝并向用户态返回 `-1`。为了打通完整调用链，分别修改了以下位置：

文件	修改内容	作用
<code>user/user.h</code>	声明 <code>interpose()</code>	允许用户程序编译调用
<code>user/usys.pl</code>	生成用户态调用桩	写入调用号并执行 <code>ecall</code>
<code>kernel/syscall.h</code>	定义 <code>SYS_interpose = 22</code>	分配唯一系统调用号

文件	修改内容	作用
kernel/syscall.c	注册处理函数并检查掩码	在真正分派前拒绝调用
kernel/sysproc.c	实现 sys_interpose()	读取参数并写入进程状态
kernel/proc.h	增加掩码和允许路径	保存每个进程的策略
Makefile	加入 _sandbox	将测试程序写入文件系统镜像

新进程在 `allocproc()` 中将策略初始化为空，避免复用 `struct proc` 时残留旧限制。`kfork()` 将父进程的 `syscall_mask` 和 `allowed_path` 复制给子进程，使限制不能通过 fork 后 exec 绕过。

4.3.2 分派前拦截

`syscall()` 先验证调用号，再检查：

```
if(p->syscall_mask & (1U << num)) {
    p->trapframe->a0 = -1;
    return;
}
```

拦截放在 `syscalls[num]()` 之前，因此被禁止的内核服务不会产生副作用。例如 `32768 = 1 << SYS_open`，执行 `sandbox 32768 - cat README` 时，`cat` 的 `open()` 立即返回失败，输出 `cat: cannot open README`。

4.4 Sandbox with allowed pathnames: 精确路径例外

第二阶段允许被掩码限制的 `open` 和 `exec` 在路径完全匹配时继续执行。`sys_interpose()` 使用 `argstr()` 将路径复制到进程私有的 `MAXPATH` 缓冲区；分派时只对 `SYS_open`、`SYS_exec` 读取第 0 个数，并用 `strcmp()` 与保存值比较。其他被掩码覆盖的系统调用仍直接失败。

调用被 mask 命中

- └─ 不是 open/exec → 拒绝
- └─ 是 open/exec
 - └─ 参数提取失败或路径不相等 → 拒绝
 - └─ 路径完全相等 → 正常分派

```
$ sandbox 32768 - cat README
cat: cannot open README

$ sandbox 32768 README grep xv6 README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6). xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu). The main purpose of xv6 is as a teaching

$ sandbox 32768 README grep xv6 x
grep: cannot open x
```

图 4-2 sandbox 对 open 的拒绝、README 路径例外和非匹配路径拒绝

图 4-2 中同为被限制的 `open()`：`README` 与允许路径相同，`grep` 可以读取；`x` 不匹配，因而仍返回失败。路径 `"-"` 不会匹配正常文件名，兼容第一阶段“全部拒绝”的语义。

4.5 Attack xv6: 利用未清零物理页

4.5.1 漏洞原理

本 Lab 编译时通过 `LAB_SYSCALL` 故意跳过 `kalloc()`、`kfree()` 和 `uvmmalloc()` 中的内存填充/清零。`secret` 进程在 8 页全局数组开头写入 "This may help."，并在偏移 16 字节处保存参数；进程退出后这些物理页被放回 freelist，但原字节仍然存在。后续进程申请到同一页时即可读取前一进程的数据，破坏进程隔离。

4.5.2 攻击实现

`attack` 使用 `sbrk()` 申请 32 页，逐字节搜索已知标记。匹配后从标记起始地址加 16 字节处读取连续字母和数字，写到标准输出。评分器保证秘密只包含大小写字母和数字，因此这一停止条件既避免越界，也不会吞入后续垃圾数据。

`secret` 退出并释放页面

- 页面回到 freelist，但内容未清零
- `attack` 通过 `sbrk` 重新申请页面
- 搜索固定标记 "This may help."
- 读取 `marker + 16` 处的秘密

```
$ secret ReportSecret7
$ attack
ReportSecret7
$
```

图 4-3 attack 从重新分配的物理页中恢复 ReportSecret7

这个漏洞表明“程序功能仍能正常运行”不代表隔离是安全的。真实内核必须在将物理页交给不同安全域前清除旧内容；仅依赖用户进程不会主动扫描内存是不成立的安全假设。

4.6 实验中遇到的问题和解决方法

本 Lab 的困难集中在系统调用跨层接入、进程策略继承和物理页残留数据的稳定复现。

最初只增加内核处理函数时，用户程序仍然无法调用新系统调用。原因是 xv6 的系统调用由用户声明、汇编桩、调用号、分派表和内核实现共同组成，缺少任意一层都会在编译或运行时失败。逐层补齐这条调用链后，程序编译成功，`sandbox_mask` 通过。

第一版 sandbox 只修改当前进程的限制状态，子进程通过 fork 后可能恢复为无限制状态。检查进程创建路径后发现，新增字段不会由地址空间复制自动继承，因此在 `kfork()` 中显式复制系统调用掩码和允许路径，`sandbox_fork` 随后通过。

加入路径例外时，若只判断掩码和字符串，可能把例外错误应用到其他系统调用，或放行前缀相同但实际不同的路径。后续只对 `open` 和 `exec` 提取第 0 个参数，并使用完整字符串精确比较，`path`、`most` 和 `minus` 三类边界测试全部通过。

实现内存残留攻击时，攻击程序偶尔无法拿到保存秘密的那一页。物理页重新分配顺序会受到 `freelist` 和先前内核分配影响，不能假定第一次 `sbrk()` 就返回目标页。因此一次申请 32 页并逐页搜索已知标记，使随机秘密测试稳定通过。

4.7 最终验收与心得

关键提交如下：

提交	内容
1c298e3	完成 GDB 问题与 <code>answers-syscall.txt</code>
ee2117e	实现系统调用掩码 <code>sandbox</code>
6cfe037	增加允许路径语义
346d204、f031d96	完成攻击程序、耗时和最终收尾

从干净状态运行 `make grade`，所有项目通过：

```
== Test answers-syscall.txt ==  
answers-syscall.txt: OK  
== Test sandbox_mask ==  
sandbox_mask: OK (4.1s)  
== Test sandbox_fork ==  
sandbox_fork: OK (0.6s)  
== Test sandbox_path ==  
sandbox_path: OK (1.3s)  
== Test sandbox_most ==  
sandbox_most: OK (0.5s)  
== Test sandbox_minus ==  
sandbox_minus: OK (1.1s)  
== Test attack ==  
attack: OK (1.0s)  
== Test time ==  
time: OK  
Score: 45/45
```

图 4-4 syscall Lab 完整评分结果 45/45

本 Lab 将上一阶段“使用系统调用”推进到“设计和拦截系统调用”。最大的收获是系统调用策略不仅是分派表中的一次判断，还必须覆盖进程初始化、fork 继承、参数复制和错误返回等完整生命周期。攻击任务进一步说明内存管理中的清零操作属于安全边界的一部分，而不是可随意删除的性能细节。

5. Lab pgtbl: Page tables

5.1 实验目的与要求

项目	内容
官方分支	pgtbl
实验主题	RISC-V Sv39 页表、地址转换、共享映射和大页
具体任务	解释进程页表、加速 getpid、递归打印页表、2 MiB superpage
基线提交	cff452f
完成提交	364eff0
实际用时	7 小时
最终评分	41/41

本 Lab 围绕虚拟地址到物理地址的映射展开。先通过现有 `pgpte()` 观察用户进程的叶子 PTE，再插入用户只读共享页；随后递归打印完整三级页表，最后把 level-1 叶子扩展为 2 MiB superpage，并处理 fork、退出和部分释放等生命周期问题。

5.2 Inspect a user-process page table: 解释用户页表

5.2.1 Sv39 地址结构与权限位

Sv39 把虚拟地址分为三级 9 位索引和 12 位页内偏移：

38	30 29	21 20	12 11	0
-----+	-----+	-----+	-----+	-----+
VPN[2]	VPN[1]	VPN[0]	page offset	
-----+	-----+	-----+	-----+	-----+

每级页表含 512 个 64 位 PTE。有效非叶子 PTE 只设置 `v`，其物理页号指向下一级页表；含 `R/W/X` 中任意一位的有效 PTE 是叶子映射。实验输出涉及的低位为：

位	含义	本实验中的用途
<code>v</code>	Valid	PTE 是否有效
<code>R/W/X</code>	读、写、执行	页面允许的操作
<code>u</code>	User	用户态能否访问
<code>a</code>	Accessed	页面是否已被访问
<code>d</code>	Dirty	页面是否已被写入

5.2.2 实际映射分析

`pgtbltest` 开头的有效页依次为两页用户只读可执行代码、数据/BSS、用户不可访问的栈保护页和用户栈。高地址处依次预留 `USYSCALL`，并映射 supervisor 可访问的 `trapframe` 和 `trampoline`。普通堆区尚未增长的地址没有有效 PTE。

虚拟地址	权限	逻辑内容
0x0、0x1000	V R X U A	程序代码及只读数据
0x2000	V R W U	data 和 BSS
0x3000	V R W	无 u 的栈保护页
0x4000	V R W U A D	用户栈
0x3fffffd000	初始无映射，后为 V R U	USYSCALL 共享页
0x3fffffe000	V R W A D	trapframe
0x3fffff000	V R X A	trampoline

```
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x217C7C5B pa 0x85F1F000 perm 0x5B
va 0x1000 pte 0x217C705B pa 0x85F1C000 perm 0x5B
va 0x2000 pte 0x217C6C17 pa 0x85F1B000 perm 0x17
va 0x3000 pte 0x217C6807 pa 0x85F1A000 perm 0x7
va 0x4000 pte 0x217C64D7 pa 0x85F19000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
...
va 0x3FFFFFD000 pte 0x217D4813 pa 0x85F52000 perm 0x13
va 0x3FFFFFE000 pte 0x217D00C7 pa 0x85F40000 perm 0xC7
va 0x3FFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
```

图 5-1 pgtbltest 的典型叶子 PTE、权限与 ugetpid 测试结果

不同运行中的物理地址会变化且不连续，这是物理页分配顺序造成的正常现象；判断页面用途应以虚拟地址、装载布局和权限位为依据。逐项解释保存在 `answers-pgtbl.txt` 中。

5.3 Speed up system calls: 共享只读 USYSCALL 页

5.3.1 设计目标

普通 `getpid()` 需要执行 `ecall` 并经历完整陷阱链。优化方案在每个进程的 `USYSCALL` 固定虚拟地址映射一页内核准备的数据，用户函数 `ugetpid()` 直接读取页首的 `struct usyscall.pid`，避免用户态/内核态切换。

5.3.2 页面生命周期

实现参照 `trapframe` 的管理方式，覆盖所有创建、失败回滚和释放路径：

`allocproc()`

- `kalloc()` 分配 `usyscall` 物理页
- 写入当前进程 `pid`
- `proc_pagetable()` 映射到 `USYSCALL`
- 权限 `PTE_R` | `PTE_U` (不含 `PTE_W`)

`freeproc()`

- `proc_freepagetable()` 解除 `USYSCALL` 映射
- `kfree()` 归还物理页

`struct proc` 保存 `struct usyscall *usyscall`。每次 `allocproc()` 已经分配新的 PID，因此在创建页表前即可初始化共享内容；`fork` 创建子进程时自然得到独立页和正确的子 PID。用户可读但不可写，避免伪造 `ugetpid()` 返回值。

同类方法还可优化 `uptime()`：内核维护一页全局 tick 计数，并把同一物理页只读映射到各用户页表。若共享数据不能原子读取，则还需序列号等一致性协议。

5.4 Print a page table: 递归打印三级映射

5.4.1 遍历算法

`vmprint()` 先打印根页表地址，再从 level 2 开始扫描 512 个 PTE。无效项跳过；有效项根据层级索引计算虚拟地址前缀，按深度输出 " .." 缩进。若 PTE 不含 `R/W/X`，它指向下一级页表并继续递归；叶子项只打印而不下降。

```
uint64 pteva = va | ((uint64)i << PXSHIFT(level));
if(level > 0 && (pte & (PTE_R | PTE_W | PTE_X)) == 0)
    vmprintwalk((pagetable_t)PTE2PA(pte), level - 1,
                pteva, depth + 1);
```

所有地址和 PTE 使用 `%p` 输出完整 64 位十六进制值。第一次编译时 GCC 的格式检查要求 `%p` 对应 `void *`，因此对数值做显式转换，既满足实验格式也通过 `-Werror`。

```

print_kpgtbl starting
page table 0x0000000085f22000
..0x0000000000000000: pte 0x00000000217c7801 pa 0x0000000085f1e000
.. ..0x0000000000000000: pte 0x00000000217c7401 pa 0x0000000085f1d000
.. .. ..0x0000000000000000: pte 0x00000000217c7c5b pa 0x0000000085f1f000
.. .. ..0x0000000000000100: pte 0x00000000217c705b pa 0x0000000085f1c000
.. .. ..0x0000000000000200: pte 0x00000000217c6cd7 pa 0x0000000085f1b000
.. .. ..0x0000000000000300: pte 0x00000000217c6807 pa 0x0000000085f1a000
.. .. ..0x0000000000000400: pte 0x00000000217c64d7 pa 0x0000000085f19000
..0x00000003fc000000: pte 0x00000000217c8401 pa 0x0000000085f21000
.. ..0x00000003fffe0000: pte 0x00000000217c8001 pa 0x0000000085f20000
.. .. ..0x00000003fffffd00: pte 0x00000000217d4853 pa 0x0000000085f52000
.. .. ..0x00000003fffffe00: pte 0x00000000217d00c7 pa 0x0000000085f40000
.. .. ..0x00000003ffffff00: pte 0x000000002000184b pa 0x0000000080006000
print_kpgtbl: OK

```

图 5-2 vmprint 输出的三级页表树及高、低地址分支

图 5-2 中根页表有索引 0 和 255 两个有效分支；最深层叶子与图 5-1 的有效页面一一对应。区别是 `print_pgtbl()` 只查询选定虚拟页并显示无效项，而 `vmprint()` 还显示指向下级页表的中间 PTE。

5.5 Use superpages: 2 MiB 大页映射

5.5.1 物理页分配器

2 MiB superpage 的物理地址必须 2 MiB 对齐。本实现从 `PHYSTOP` 以下预留 16 个连续对齐区域，共 32 MiB，建立独立的 `supermem.freelist` 和自旋锁。普通 `freerange()` 只加入 `SUPERBASE` 以下的 4 KiB 页，从根本上避免同一物理内存同时出现在两种 `freelist` 中。

`superalloc()` 和 `superfree()` 每次取出或归还一个完整 2 MiB 区域，并保留 xv6 用填充值发现悬空引用的习惯。16 个区域可以覆盖测试中父子进程同时持有的 superpage，同时为普通页、页表和内核对象保留足够内存。

5.5.2 映射与地址转换

`uvmalloc()` 遍历新增范围时，只有虚拟地址 2 MiB 对齐且剩余长度至少 2 MiB 才调用 `superalloc()`。`supermappage()` 在 level-1 PTE 直接设置物理页号及 `V|R|W|U`，不再创建 level-0 页表，因而一个 PTE 代替 512 个普通页 PTE。

原 `walk()` 默认叶子只在 level 0。实现将其重构为能返回叶子层级的 `walkleaf()`；`walkaddr()` 若遇到 level-1 叶子，还要加入虚拟地址在 2 MiB 范围内的偏移，否则内核对 superpage 内不同 4 KiB 子区的访问都会错误指向 同一物理起始页。

5.5.3 fork、释放与降级

生命周期操作	处理方式
fork	<code>uvmcopy()</code> 为子进程分配新 2 MiB 区域，复制完整内容并保持 level-1 叶子
完整释放	<code>uvmunmap()</code> 清除 level-1 PTE，调用 <code>superfree()</code>
部分释放	先降级成 512 个独立 4 KiB 页，再精确释放目标页
退出	<code>uvmfree()</code> 遍历地址空间，完整归还剩余 superpage

部分释放是本任务的关键边界。如果 `sbrk(-PGSIZE)` 只删除 level-1 PTE，会把 仍应有效的约 2 MiB 数据一起丢失；如果继续保留大页，被释放的最后 4 KiB 又 仍可访问。`superdemote()` 因此先分配 level-0 页表和 512 个普通物理页，逐页 复制原内容并继承权限，成功后再归还原 superpage，最后按普通页路径解除目标 映射。分配失败时会释放已创建的小页并保留原映射，避免半降级状态。

```
superpg_fork starting
usertrap(): unexpected scause 0xf pid=69
           sepc=0x2ec stval=0x5001
superpg_fork: OK
superpg_free starting
usertrap(): unexpected scause 0xd pid=70
           sepc=0x40a stval=0xffff001
superpg_free: OK
pgtbltest: all tests succeeded
```

图 5-3 superpage 在 fork、完整释放和部分降级场景下全部通过

图 5-3 的两次 `unexpected scause` 是测试子进程故意读取已经释放的地址；子进程因页错误退出正是预期结果，所以其后的 `superpg_fork` 和 `superpg_free` 均为 `OK`。

5.6 实验中遇到的问题和解决方法

页表实验的问题通常不会停留在建立映射这一处，而会延伸到权限、地址翻译、fork、释放和失败回滚。

`ugetpid` 最初一调用就触发 load page fault。虽然代码已经定义 `USYSCALL` 虚拟地址，但进程创建时并没有分配物理页并建立映射。补齐物理页分配、用户只读映射、失败回滚和退出释放后，连续 64 次 fork 得到的 PID 都与普通 `getpid()` 一致。

编写 `vmprint` 时，使用 `%p` 打印整数形式的虚拟地址和页表项会被 `-Werror` 拒绝，因为格式检查要求参数确实为指针。将 VA、PTE 和 PA 显式转换为 `void *` 后，输出格式和专项评分均通过。

支持 superpage 后，普通 `walkaddr()` 若仍按 4 KiB 叶子处理，会丢失 2 MiB 大页内部的偏移。解决方法是让 `walkleaf()` 同时返回叶子所在层级，再根据页大小补上页内偏移，数据写入和读取检查通过。

另一个问题是释放大页中的 4 KiB 区间。level-1 PTE 只能描述整张 2 MiB 大页，直接清除会误删其余仍有效内容。实现先把大页降级为 512 个普通页，再按原有路径释放目标页。fork 时也不能复用只复制 4 KiB 的普通逻辑，而是为子进程独立分配并复制完整 2 MiB。部分释放、内容保持和 `superpg_fork` 测试均通过。

5.7 最终验收与心得

关键提交如下：

提交	内容
6097a95	完成用户页表逐项解释
781903e	映射 USYSCALL 并加速 getpid
d8c29ac	实现递归 vmprint
b7b1643	实现 superpage 分配、复制、释放和降级
364eff0	记录 7 小时实际用时并完成验收

先运行 `pgtbltest` 验证页表结构和 superpage，再执行完整评分：

```
== Test pgtbltest ==  
$ make qemu-gdb  
== Test pgtbltest: ugetpid ==  
pgtbltest: ugetpid: OK  
== Test pgtbltest: print_kpgtbl ==  
pgtbltest: print_kpgtbl: OK  
== Test pgtbltest: superpg ==  
pgtbltest: superpg: OK  
== Test answers-pgtbl.txt ==  
answers-pgtbl.txt: OK  
== Test usertests ==  
$ make qemu-gdb  
== Test time ==  
time: OK  
Score: 41/41
```

图 5-4 pgtbl Lab 完整评分结果 41/41

make grade 中 ugetpid、print_kpgtbl、superpg、答案、usertests 和耗时 检查全部通过，最终为 41/41。本 Lab 的主要收获是页表修改不能只考虑“建立一个 PTE”，还必须保持地址翻译、权限、fork、收缩、退出和失败回滚的一致性。superpage 的代码量相对有限，但部分释放揭示了内存管理中粒度变化所带来的复杂资源生命周期。

6. Lab traps: Traps

6.1 实验目的与要求

项目	内容
官方分支	traps
实验主题	RISC-V 汇编、内核栈、陷阱帧和时钟中断
具体任务	RISC-V assembly、Backtrace、Alarm
基线提交	4270ccc
完成提交	895895c
实际用时	5 小时
最终评分	95/95

本 Lab 从静态反汇编逐步进入运行时陷阱处理。首先分析 RISC-V 调用约定、编译器内联和大小端表示；随后沿内核栈帧恢复系统调用的调用链；最后增加两个系统调用，利用时钟中断把用户进程临时重定向到 alarm handler，并在处理结束后完整恢复现场。

6.2 RISC-V assembly: 调用约定与反汇编分析

6.2.1 参数传递与编译优化

RISC-V 使用 `a0` 至 `a7` 传递前八个整数或指针参数。`main()` 调用 `printf()` 时，`a0` 保存格式字符串地址，`a1=12`，`a2=13`。其中 `f(8)+1` 没有产生函数调用：编译器先内联 `f()`，再把结果常量折叠为 `12`；`g()` 同样被内联为一条执行 `x+3` 的 `addiw` 指令。

当前工具链生成的 `user/call.asm` 中，`printf` 位于 `0x6f6`，调用指令位于 `0x30`。RISC-V 的 `jal` 把下一条指令地址写入 `ra`，因此进入 `printf` 后 `ra=0x34`。

6.2.2 大小端与未定义行为

整数 `0x00646c72` 在小端机器中的内存字节依次为 `72 6c 64 00`，解释为字符串 `"rld"`，所以程序输出 `He110 World`。若改为大端机器且仍要得到相同字符串，整数应改为 `0x726c6400`；十进制数 `57616` 不需要修改，因为 `%x` 格式化的是数值而不是内存字节序列。

当格式串要求两个整数而调用者只提供一个时，第二个值没有确定答案。`printf` 会读取未提供的参数位置，程序属于未定义行为，不能把某次运行中偶然出现的数值写成语言或 ABI 保证的结果。

```

● lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$ ./grade-lab-traps answers-traps.txt
make: 'kernel/kernel' is up to date.
== Test answers-traps.txt ==
answers-traps.txt: OK

```

图 6-1 RISC-V 汇编分析答案通过专项检查

6.3 Backtrace: 遍历内核栈帧

6.3.1 栈帧布局与边界

编译参数保留了帧指针。RISC-V 内核函数进入后，`s0` 指向当前栈帧顶部，`s0-8` 保存返回地址，`s0-16` 保存上一帧的帧指针。因此回溯过程可表示为：

```

r_fp() 读取 s0
→ *(fp - 8) 取得当前返回地址
→ *(fp - 16) 取得调用者 fp
→ 重复，直到离开当前 4 KiB 内核栈页

```

在 `kernel/riscv.h` 中用内联汇编 `mv` 读取 `s0`，在 `kernel/printf.c` 中实现 `backtrace()`。初始 `fp` 向下调整得到当前内核栈页下界，并以上界 `stack_bottom + PGSIZE` 限制遍历；同时要求 `fp` 至少高于页底 16 字节，保证 读取返回地址和上一帧指针不会跨入 guard page。

6.3.2 调用链验证

`sys_pause()` 调用 `backtrace()` 后，`bttest` 打印三个返回地址。使用 `riscv64-linux-gnu-addr2line -e kernel/kernel` 解析后得到：

返回地址	源码位置	含义
0x80001e3e	kernel/sysproc.c:75	<code>sys_pause()</code> 调用点
0x80001d18	kernel/syscall.c:141	系统调用分派
0x80001a9c	kernel/trap.c:80	用户陷阱入口

```

● lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$ ./grade-lab-traps backtrace
make: 'kernel/kernel' is up to date.
== Test backtrace test == backtrace test: OK (0.8s)
● lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$ for addr in $(grep -oE '^0x00000000[0-9a-f]+' xv6.out); do
    printf '%s -> ' "$addr"
    riscv64-linux-gnu-addr2line -e kernel/kernel "$addr"
done
0x0000000080001e3e -> /home/lbyy/xv6-labs-2025/kernel/sysproc.c:75
0x0000000080001d18 -> /home/lbyy/xv6-labs-2025/kernel/syscall.c:141 (discriminator 1)
0x0000000080001a9c -> /home/lbyy/xv6-labs-2025/kernel/trap.c:80
○ lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$

```

图 6-2 Backtrace 专项评分及三个返回地址的源码定位

该结果同时验证了栈遍历格式和真实调用链。实现中 `%p` 的参数显式转换为 `void *`，满足内核 `printf` 的格式检查并保持 64 位地址完整输出。

6.4 Alarm：由时钟中断进入用户处理函数

6.4.1 系统调用与进程状态

实验新增 `sigalarm(interval, handler)` 和 `sigreturn()`，分配系统调用号 22、23，并补齐用户声明、调用桩、内核分派表和处理函数。 `struct proc` 保存：

字段	作用
<code>alarm_interval</code>	两次 alarm 之间需要经过的 timer ticks
<code>alarm_ticks</code>	当前周期已经累计的 ticks
<code>alarm_handler</code>	用户处理函数入口地址
<code>alarm_active</code>	handler 是否正在运行，防止重入
<code>alarm_trapframe</code>	handler 执行前的完整用户寄存器现场

`sigalarm()` 更新配置并重新计时。进程退出或结构被复用时清空 alarm 状态；fork 复制父进程的周期和 handler，但子进程从未激活、计数为零的状态开始，避免继承一半执行中的 handler 现场。

6.4.2 中断触发与现场恢复

`usertrap()` 识别到 timer interrupt 后累计当前进程的 ticks。当累计值达到周期且 `alarm_active` 为 0 时，内核先复制整个 trapframe，再把返回用户态所用的 `epc` 改为 handler 地址：

```
p->alarm_trapframe = *(p->trapframe);
p->trapframe->epc = p->alarm_handler;
```

于是 `prepare_return()` 不再回到被中断指令，而是首次进入用户 handler。handler 调用 `sigreturn()` 后，内核恢复保存的完整 trapframe 并清除 active 标记，使原程序从被中断位置继续执行。

完整恢复比只恢复 `epc` 更重要：时钟中断可能发生在任意用户指令之间，通用寄存器、栈指针和参数寄存器都属于被中断程序的现场。`sys_sigreturn()` 还先保存原 trapframe 中的 `a0` 并将其作为系统调用返回值，否则统一系统调用分派代码会把新的返回值再次写入 `a0`，覆盖刚恢复的原始寄存器。

`alarm_active` 防止 handler 自身运行时间超过一个周期时再次进入 handler；只有显式执行 `sigreturn()` 后，下一次 alarm 才能被投递。


```

1byy@LAPTOP-US2J300J:~/xv6-labs-2025$ ./grade-lab-traps alarm
grep -E 'test[0-3] passed|FAIL|exec alarmtest failed' xv6.out
make: 'kernel/kernel' is up to date.
== Test running alarmtest == (4.2s)
== Test alarmtest: test0 ==
alarmtest: test0: OK
== Test alarmtest: test1 ==
alarmtest: test1: OK
== Test alarmtest: test2 ==
alarmtest: test2: OK
== Test alarmtest: test3 ==
alarmtest: test3: OK
test0 passed
test1 passed
test2 passed
test3 passed

```

图 6-3 Alarm 的单次/重复触发、寄存器恢复、防重入和 a0 保持测试

6.5 实验中遇到的问题和解决方法

本 Lab 同时涉及编译器格式检查、文件系统镜像、内核栈和异步寄存器现场，问题表现相似但根因位于不同层次。

Backtrace 中用 `%p` 直接打印整数返回地址时触发 `-Werror=format`。将地址显式转换为 `void *` 后通过编译。随后用于解析返回地址的命令又因为 Ubuntu 未安装 `rg` 而失败，最终改用 `grep -oE` 提取地址，再通过 `while read` 逐个调用 `addr2line`，三个返回地址都成功定位到源码行。

实现 Alarm 后曾出现 `exec alarmtest failed`。问题不在系统调用本身，而是 `_alarmtest` 尚未加入当前分支的 `UPROGS`，旧 `fs.img` 中也没有这个程序。修改 Makefile 并重建文件系统镜像后，测试程序可以正常执行。

第一版 Alarm 只把 `epc` 改到 handler，没有完整保存用户寄存器；handler 返回后，原程序的寄存器和 `a0` 会被破坏。时钟中断还可能在 handler 执行期间再次投递，造成重入。后续保存完整 trapframe，并用 `alarm_active` 阻止重复进入，只有 `sigreturn()` 恢复现场后才解除。test1、test2 和 test3 均通过。

6.6 最终验收与心得

关键提交如下：

提交	内容
c572d63	完成 RISC-V 汇编分析题

提交	内容
59b7b78	实现内核栈 Backtrace
5abddb9	接入 Alarm 系统调用并处理时钟中断
895895c	记录 5 小时实际用时并完成验收

从干净状态运行 `make grade`，汇编答案、Backtrace、Alarm 的 test0 至 test3、`usertests` 和时间检查全部通过：

```
== Test answers-traps.txt ==
answers-traps.txt: OK
== Test backtrace test ==
$ make qemu-gdb
backtrace test: OK (2.1s)
== Test running alarmtest ==
$ make qemu-gdb
(3.6s)
== Test  alarmtest: test0 ==
alarmtest: test0: OK
== Test  alarmtest: test1 ==
alarmtest: test1: OK
== Test  alarmtest: test2 ==
alarmtest: test2: OK
== Test  alarmtest: test3 ==
alarmtest: test3: OK
== Test usertests ==
$ make qemu-gdb

usertests: OK (41.0s)
== Test time ==
time: OK
Score: 95/95
```

图 6-4 traps Lab 完整评分结果 95/95

本 Lab 把静态 ABI 知识与动态控制流联系起来：函数调用依赖寄存器和栈帧约定，系统调用通过 `trapframe` 保存跨特权级现场，而异步 `alarm` 又要求内核在未知指令边界上完整保存并恢复用户状态。最大的收获是，中断处理的正确性不仅取决于“能跳到 handler”，更取决于恢复后程序是否像从未被打断一样继续运行。

7. Lab cow: Copy-on-write

7.1 实验目的与要求

项目	内容
官方分支	<code>cow</code>
实验主题	写时复制 fork、页错误和共享物理页生命周期
具体任务	Implement copy-on-write fork
基线提交	<code>cf0eb5b</code>
完成提交	<code>7fc6398</code>
实际用时	4 小时
最终评分	130/130

原始 `fork()` 通过 `uvmcopy()` 为子进程逐页分配物理内存并复制父进程内容。当父进程占用超过一半物理内存时，即使子进程马上调用 `exec()`，`fork` 也可能因无法再复制一份完整地址空间而失败。COW 把复制推迟到父子中的某一方首次写入，没有被写过的页面始终共享，从而同时减少 `fork` 延迟和内存峰值。

7.2 COW fork: 共享父进程物理页

7.2.1 PTE 状态设计

父子共享页面后必须清除 `PTE_W`，使 CPU 在写入时产生 store page fault。但“因 COW 临时只读”和“代码段原本只读”不能混为一谈，否则写代码段也会被错误升级为可写。本实现使用 RISC-V PTE 的 RSW 软件保留位记录：

```
#define PTE_COW (1L << 8)
```

只有原 PTE 含 `PTE_W` 时，`uvmcopy()` 才清除写权限并设置 `PTE_COW`；代码段等原生只读页不设置该标志，非法写入仍会终止进程。`PTE_COW` 位属于 `PTE_FLAGS()` 保留的低 10 位，可随映射复制而不会影响硬件地址转换。

7.2.2 fork 映射流程

修改后的 `uvmcopy()` 不再调用 `kalloc()` 和 `memmove()` 复制每个用户页，而是把同一物理地址映射进子页表：

遍历父进程有效 PTE

- 原页面可写：父 PTE 清 W、设置 COW
- 子页表映射同一 PA 和相同只读/COW 权限

- 物理页引用计数加 1
- 完成后刷新当前 hart 的 TLB

父 PTE 也必须变为只读，否则父进程可以绕过 page fault 直接修改共享页面。修改页表后调用 `sfence_vma()` 清除旧的可写 TLB 项，保证后续写入依据新权限 重新查询页表。

7.3 写缺页：按需创建私有副本

`usertrap()` 已把 load/store page fault 交给 `vmfault()`。本实现保留原有懒分配能力，并增加对“有效、用户可访问、含 PTE_COW 的写故障”的处理：

```
store page fault
├── PTE 不是 COW → 非法写入，返回失败并终止进程
└── PTE 是 COW
    ├── 引用计数为 1 → 直接清 COW、恢复 PTE_W
    └── 引用计数大于 1
        → kalloc() 新页
        → 复制原页全部 4096 字节
        → 当前 PTE 改指新页并恢复写权限
        → 旧页引用计数减 1
```

引用数为 1 时说明其他共享者已经退出或拆分页，当前进程已是唯一所有者，不必进行无意义的物理复制。若 `kalloc()` 失败，`vmfault()` 返回 0，用户陷阱路径会终止当前进程，而不会让它继续写共享页面。更新 PTE 后再次执行 `sfence_vma()`，使故障指令重试时看到新的可写映射。

7.4 copyout：处理内核发起的用户内存写入

用户指令写只读页会触发硬件 page fault，但 `read()`、管道和文件系统等内核代码通过 `copyout()` 直接写用户物理地址，不会触发用户态 store fault。如果仍按旧逻辑处理，子进程读管道时会直接修改与父进程共享的 `buf` 页面。

因此 `copyout()` 在每页写入前检查 PTE：普通可写页直接复制；COW 页调用同一个 `vmfault(..., read=0)` 路径先取得私有页；原生只读页返回 -1。用户缺页和内核代写共享同一套拆分规则，避免两份实现产生权限或引用计数差异。`cowtest` 的 `file` 项专门验证该路径，子进程收到管道数据后，父进程缓冲区仍保持原值。

7.5 物理页引用计数与并发

共享映射使一个物理页可能同时出现在多个用户页表中，原有 `uvmunmap()` 每次都直接 `kfree()` 会导致第一个进程退出时过早回收页面。实现增加固定引用计数数组：

```
index = (pa - KERNBASE) / PGSIZE
kalloc()      → count = 1
COW fork 映射 → count++
解除一次映射 → count--
count == 0    → 填充垃圾值并放回 freelist
```

数组覆盖 [KERNBASE, PHYSTOP) 内所有可分配页，由独立的 `krefs` 自旋锁保护。初始化 freelist 时先把每页引用数设为 1，再通过统一的 `kfree()` 路径递减并加入空闲链表；以后所有内核页和用户页都遵守同一计数不变式。引用数大于零时 `kfree()` 只递减而不破坏页面内容。

锁使多个进程同时 fork、退出和拆分同一共享页时，递增与递减不会丢失。测试中的 `forkfork` 连续创建多层子进程并让它们交错退出，既检查引用计数竞争，也检查父进程的共享内容始终未被修改。

7.6 实验中遇到的问题解决方法

COW 的难点在于同一物理页会同时出现在多个进程页表中，局部修复某一次写缺页并不能保证 fork、copyout 和退出路径正确。

原始 fork 会立即复制全部物理页，大地址空间的内存峰值接近两倍，`simple` 测试会因内存不足失败。改为父子共享物理页、清除写权限并标记 `PTE_COW` 后，复制被延迟到首次写入，两次 `simple` 均通过。标记时只处理原本可写的页面，避免把代码段等真正只读页面错误变成可写页。

加入共享映射后，子进程退出可能导致父进程页面被提前放回 freelist。原因是普通 `kfree()` 不知道还有其他页表引用该页。为每个物理页增加带锁引用计数，只有计数降为零时才真正释放，`three` 和 `forkfork` 的交错退出测试随后通过。

用户 store 会产生 COW page fault，但内核 `copyout()` 直接访问物理地址，不经过相同 trap 路径，最初会绕过拆分逻辑。将拆页操作提取为可复用函数，并在 copyout 写入前显式调用后，file 和 copyout 测试通过。修改父子 PTE 权限后还执行 `sfence_vma()`，避免 TLB 继续使用旧的可写权限。

7.7 最终验收与心得

关键提交如下：

提交	内容
<code>ecafd04</code>	实现 COW 映射、写缺页、copyout 和引用计数
<code>7fc6398</code>	记录 4 小时实际用时并完成验收

从干净状态运行 `make grade`，`cowtest` 的 `simple`、`three`、`file`、`forkfork`，`usertests` 的 `copyin`、`copyout` 和全量回归，以及时间检查全部通过：

```
$ make qemu-gdb
(25.4s)
== Test    simple ==
    simple: OK
== Test    three ==
    three: OK
== Test    file ==
    file: OK
== Test    forkfork ==
    forkfork: OK
== Test    usertests ==
$ make qemu-gdb
(36.6s)
== Test    usertests: copyin ==
    usertests: copyin: OK
== Test    usertests: copyout ==
    usertests: copyout: OK
== Test    usertests: all tests ==
    usertests: all tests: OK
== Test time ==
time: OK
Score: 130/130
```

图 7-1 COW Lab 完整评分结果 130/130

本 Lab 的核心不只是“发生错误时复制一页”，而是建立共享页从创建、并发引用、按需拆分到最后回收的完整生命周期。页表权限负责把普通写操作转化为内核可见的事件，RSW 位保存软件语义，引用计数保证资源存活，TLB 刷新保证硬件及时采用新权限；其中任一环节缺失都会表现为隔离破坏、内存泄漏或悬空映射。

8. Lab net: Network driver

8.1 实验目的与要求

项目	内容
官方分支	net
实验主题	E1000 网卡驱动、以太网/IP/UDP 接收和阻塞式端口队列
具体任务	Part One: NIC、Part Two: UDP Receive
基线提交	982b43b
完成提交	d9ad0dc
实际用时	6 小时
最终评分	171/171

本 Lab 从设备驱动和协议栈两个层次建立 xv6 的网络接收路径。第一部分通过 E1000 发送、接收描述符环在内核与模拟网卡之间转移以太网帧；第二部分解析 IPv4/UDP 报文，按目标端口排队，并通过 `bind()`、`recv()` 和 `unbind()` 向用户进程提供阻塞式数据报接口。实现的核心问题是缓冲区所有权：同一页在驱动、协议栈、端口队列和用户接收调用之间流转时，任一分支都必须明确由谁最终释放。

8.2 Part One: E1000 NIC 驱动

8.2.1 实验目的

补全 `kernel/e1000.c` 中的发送与接收逻辑，使 xv6 能通过 QEMU 模拟的 Intel E1000 网卡发送以太网帧，并在网卡中断中取出所有已经到达的帧交给 `net_rx()`。同时保证 16 项环形描述符被循环复用，设备未完成 DMA 时不会覆盖仍在使用的槽位。

8.2.2 描述符环、DMA 与调用链

E1000 的 TX/RX 环各包含 16 个描述符。描述符中的 `addr` 指向一页内核缓冲区，设备通过 DMA 直接读取待发送帧或写入接收帧；软件与设备分别通过 `head/tail` 寄存器协作，因此移动 `tail` 不只是修改普通变量，而是在向设备移交描述符所有权。

发送: `sys_send()/arp_rx()`

- `e1000_transmit()`
- 检查 TDT 指向描述符的 DD 位
- 填入物理缓冲区地址、长度和 EOP|RS
- 推进 E1000_TDT，设备 DMA 发送

接收: E1000 写入 RX 缓冲区并触发中断

- e1000_intr() 确认中断
- e1000_recv() 从 RDT 后一项开始检查 DD
- 为描述符安装新页并推进 E1000_RDT
- net_rx() 处理旧页中的以太网帧

DD 表示设备已经完成该描述符, EOP 表示当前缓冲区包含一帧的末尾, RS 要求发送完成后回写状态。填好描述符后执行内存屏障, 再更新 tail 寄存器, 避免设备看到 tail 已推进但描述符字段尚未全部可见。

8.2.3 设计与实现步骤

发送路径由 e1000_lock 串行化, 因为多个进程可能同时调用 send(), ARP 回复也可能在中断处理路径提交发送请求。实现先读取 E1000_TDT, 若当前描述符没有 DD 则返回 -1, 由调用者释放尚未移交的页; 若描述符可用, 则释放该槽上一次已经发送完成的缓冲区, 填入本次帧并以模 16 推进 tail。这样旧页只在硬件确认完成后回收, 新页在发送成功后由驱动负责。

接收路径使用循环而不是每次中断只处理一个描述符。每轮查看 $(E1000_RDT + 1) \% RX_RING_SIZE$: 若没有 DD 就停止; 否则先保存旧页和帧长, 再分配替换页、清状态并归还描述符。只有带 EOP 且无硬件错误的完整帧才交给 net_rx(), 其余帧直接释放。若暂时无法分配替换页, 则丢弃当前帧并继续让原页留在接收环中, 避免把空地址交给设备或使环永久停顿。

8.2.4 实验结果

专项运行 ./grade-lab-net txone arp_rx ip_rx 后三项均为 OK。同时读取 packets.pcap, 可以看到 xv6 发出的 UDP 包、主机查询 xv6 地址的 ARP Request、xv6 返回的 ARP Reply, 以及主机向 xv6 发送的 UDP 包, 证明发送 DMA、接收中断和双向链路均已打通。

```

lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$ ./grade-lab-net txone arp_rx ip_rx
tcpdump -nn -r packets.pcap 2>/dev/null | sed -n '1,4p'
make: 'kernel/kernel' is up to date.
== Test running nettest == (10.0s)
== Test nettest: txone ==
nettest: txone: OK
== Test nettest: arp_rx ==
nettest: arp_rx: OK
== Test nettest: ip_rx ==
nettest: ip_rx: OK
11:47:02.109118 IP 10.0.2.15.2003 > 10.0.2.2.26099: UDP, length 5
11:47:02.109682 ARP, Request who-has 10.0.2.15 tell 10.0.2.2, length 46
11:47:02.110324 ARP, Reply 10.0.2.15 is-at 52:54:00:12:34:56, length 28
11:47:02.167433 IP 10.0.2.2.34312 > 10.0.2.15.2001: UDP, length 5

```

图 8-1 E1000 NIC 专项评分与抓包结果

8.2.5 分析讨论

描述符环的不变式是：软件只能填写带 `DD` 的 TX 描述符，只能消费带 `DD` 的 RX 描述符；tail 推进后，对应槽位在设备完成前不能再次使用。发送成功意味着 缓冲页所有权已经交给驱动，而不是意味着页面可以立即释放；接收则必须先为设备 补上替换页，才能把旧页移交协议栈。锁、状态位、内存屏障和模运算分别解决并发 提交、设备完成通知、CPU/设备可见顺序和环形回绕问题，四者缺一不可。

8.3 Part Two: UDP Receive

8.3.1 端口绑定与队列设计

内核建立 64 项静态端口表，每个已绑定端口维护独立 FIFO，最多缓存 16 个数据包。所有绑定状态、队首队尾和计数由 `netlock` 保护。静态上限避免网络输入无限占用 物理页；独立队列则防止某个繁忙端口阻塞其他端口。重复 `bind(port)` 保持幂等，`unbind(port)` 先在锁内撤销绑定、摘下队列并唤醒等待者，再在锁外逐页释放，避免 长时间持锁。

为了不再为链表节点额外申请物理页，`struct udp_packet` 元数据存放在接收缓冲页 末尾。E1000 配置的帧缓冲上限约为 2 KiB，而页面为 4 KiB；入队前仍显式检查 帧长不会覆盖页尾元数据。节点记录源 IP、源端口、负载偏移和长度，数据本身继续 留在原 RX 页中，实现从驱动到用户进程的一页式所有权转移。

8.3.2 IPv4/UDP 接收与阻塞唤醒

`ip_rx()` 按顺序验证以太网帧边界、IPv4 版本、可变 IP 头长、IP 总长度、UDP 协议号和 UDP 长度。网络字段使用 `ntohs()`、`ntohl()` 转为主机字节序。畸形包、非 UDP 包、未绑定端口以及队列已满的包统一进入释放路径；有效包按目标端口入队，随后以队列地址为 `channel` 调用 `wakeup()`。

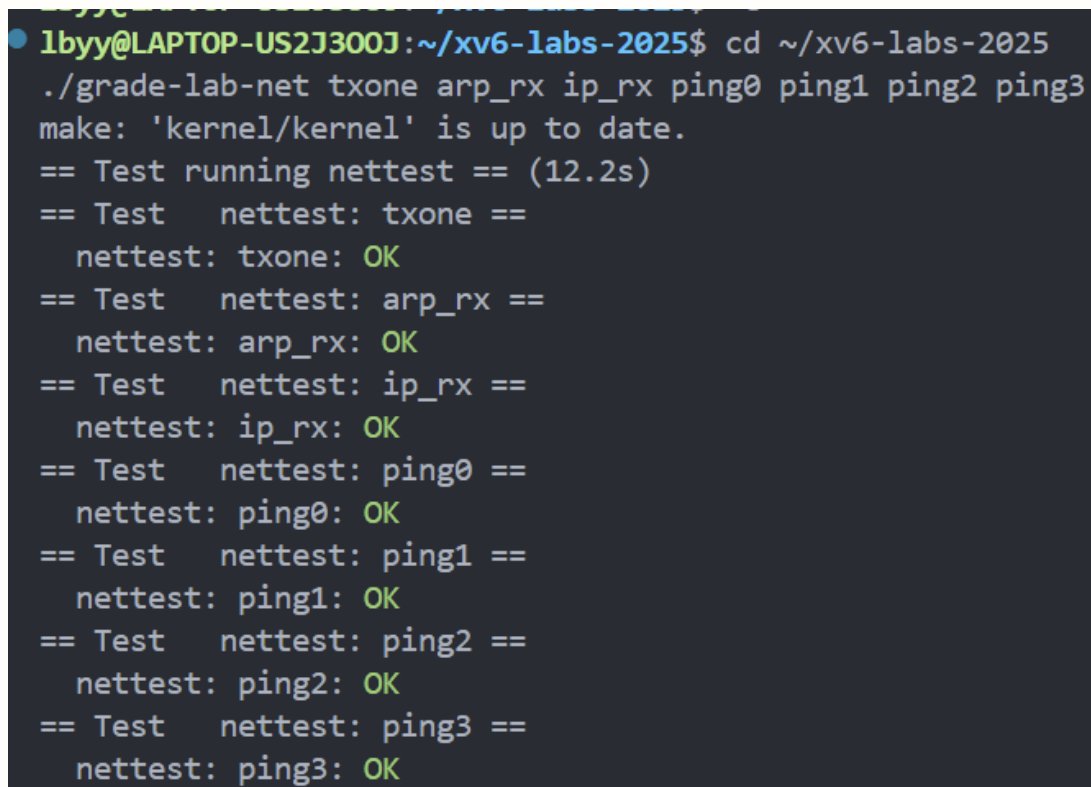
`sys_recv()` 在持有同一把 `netlock` 时检查队列。队列为空时调用 `sleep(queue, &netlock)`，该接口以原子方式释放锁并睡眠，避免在“检查为空”和“进入睡眠”之间丢失网络中断的唤醒。出队后先释放锁，再用 `copyout()` 返回源 地址、源端口和不超过 `maxlen` 的 UDP 负载，最后释放数据包页。若进程被杀死、端口已解绑或任一用户地址复制失败，则返回 `-1`，同时保证已出队页面不会泄漏。

E1000 RX 中断 → `net_rx()` → `ip_rx()`
→ 校验 IPv4/UDP 头与长度
→ 按 `dport` 查找端口队列
→ FIFO 入队并 `wakeup(queue)`
→ `sys_recv()` 被唤醒并出队
→ `copyout()` 返回源信息和 UDP payload
→ `kfree()` 释放接收页

8.3.3 资源回收与回归验证

发送侧同时补齐失败回收：`sys_send()` 和 ARP 回复只有在 `e1000_transmit()` 成功后才把缓冲页所有权交给驱动，TX 环已满时由调用者立即 `kfree()`。接收侧的所有丢包分支、`copyout()` 失败和解绑清队列也都释放相应页面。完整 `nettest` 中的 `free` 项用于检查网络测试结束后空闲物理页数量，能够发现这些低频错误路径上的泄漏。

七项发送、ARP、IP 与多轮 ping 回归全部通过：



```
lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$ cd ~/xv6-labs-2025
./grade-lab-net txone arp_rx ip_rx ping0 ping1 ping2 ping3
make: 'kernel/kernel' is up to date.
== Test running nettest == (12.2s)
== Test    nettest: txone ==
    nettest: txone: OK
== Test    nettest: arp_rx ==
    nettest: arp_rx: OK
== Test    nettest: ip_rx ==
    nettest: ip_rx: OK
== Test    nettest: ping0 ==
    nettest: ping0: OK
== Test    nettest: ping1 ==
    nettest: ping1: OK
== Test    nettest: ping2 ==
    nettest: ping2: OK
== Test    nettest: ping3 ==
    nettest: ping3: OK
```

图 8-2 UDP Receive 七项回归评分

8.4 实验中遇到的问题和解决方法

Network Lab 的问题横跨 DMA 描述符、硬件中断、协议解析、睡眠唤醒和宿主网络环境。定位时必须先区分内核实现错误与外部网络差异。

TX 描述符提交后，缓冲页仍可能被设备 DMA 读取，若立即释放会产生悬空地址；若在环回绕时忘记释放，又会泄漏。最终只在槽位重新出现 `DD`、确认设备完成后释放上一页，`txone` 和 `free` 测试通过。接收侧最初一次中断只处理一个描述符，会让连续完成的包滞留在环中，因此改为循环消费所有带 `DD` 的描述符，ARP 和 IP 接收测试通过。

UDP 接收队列为空时，如果检查队列和进入睡眠不是原子过程，中断可能恰好在两者之间到达，造成丢失唤醒。后续用同一把 `netlock` 保护检查与 `sleep/wakeup`，多轮 ping 测试稳定通过。

协议头中的长度来自不可信网络输入，直接按字段解析可能越过实际帧边界。实现按层 校验 IP 版本、头长、总长、UDP 长度和收到的帧长；同时明确缓冲页所有权，在 TX 失败、畸形包、端口解绑和 copyout 失败路径统一释放，最终 `free: OK`。

首次完整评分时，所有收发功能已经正常，但 DNS 项得到的地址与评分脚本不一致。检查后发现宿主代理把域名解析为代理保留地址，而不是测试期待的真实地址。临时 关闭代理后从干净状态重新评分，`dns` 通过并得到 171/171。这个问题没有通过修改 内核伪造结果，而是修正了外部测试环境。

首次完整评分时，协议栈已经正确收到了 DNS 响应，但宿主代理把 `pdos.csail.mit.edu` 解析为 `198.18.0.158`，与测试固定期待的 `128.52.129.126` 不同。关闭代理后重新运行 `make grade`，DNS、内存释放和时间 检查均通过，最终得分为 171/171。这一过程也说明网络实验需要区分内核协议实现错误和宿主网络环境差异，不能为了迎合测试在内核中伪造 DNS 结果。

8.5 最终验收与心得

关键提交如下：

提交	内容
<code>3edc8d7</code>	实现 E1000 发送、接收描述符环和缓冲区所有权管理
<code>8096c80</code>	实现 UDP 端口队列、阻塞接收、报文校验和失败回收
<code>d9ad0dc</code>	记录 6 小时实际用时并完成验收

```
$ make qemu-gdb
(12.4s)
== Test   nettest: txone ==
nettest: txone: OK
== Test   nettest: arp_rx ==
nettest: arp_rx: OK
== Test   nettest: ip_rx ==
nettest: ip_rx: OK
== Test   nettest: ping0 ==
nettest: ping0: OK
== Test   nettest: ping1 ==
nettest: ping1: OK
== Test   nettest: ping2 ==
nettest: ping2: OK
== Test   nettest: ping3 ==
nettest: ping3: OK
== Test   nettest: dns ==
nettest: dns: OK
== Test   nettest: free ==
nettest: free: OK
== Test time ==
time: OK
Score: 171/171
```

图 8-3 Network Lab 完整评分结果 171/171

本 Lab 将并发同步和资源生命周期落实到了硬件与软件的边界：描述符状态决定 CPU 与网卡何时能够重新使用同一槽位，端口队列决定中断生产者和进程消费者如何交接数据，页面所有权则贯穿 DMA、协议解析、排队、复制和释放全过程。相比只让正常路径通过，完整实现更重要的是让环满、内存不足、畸形包、解绑和复制失败等路径仍保持状态一致且不泄漏。

9. Lab lock: Locking

9.1 实验目的与要求

项目	内容
官方分支	lock
实验主题	多核内存分配、锁竞争和写者优先读写锁
具体任务	Memory allocator、Read-write lock
基线提交	af12b48
完成提交	17aa0bc
实际用时	7 小时
最终评分	100/100

本 Lab 关注的不是简单地“加锁保证正确”，而是在保持共享状态不变式的同时提高多核并行度。第一项把全局物理页空闲链表拆成每 CPU 独立链表，使常见的本地分配和释放互不竞争；第二项实现写者优先的读写自旋锁，让多个只读临界区并发，同时避免持续到来的读者使写者优先长期饥饿。

9.2 Memory allocator: 每 CPU 空闲链表

9.2.1 实验目的

原始 `kalloc()` 和 `kfree()` 共用唯一的 `kmem.freelist` 和 `kmem.lock`。多核上的进程频繁扩张、收缩地址空间时，即使它们操作的是不同物理页，也必须在同一把锁上串行。`kalloc_test` 通过统计 `acquire()` 中 test-and-set 失败次数量化这种竞争。本任务要求建立每 CPU 空闲链表，并在本地链表耗尽时安全地从其他 CPU 偷取页面。

9.2.2 数据结构与本地快速路径

分配器改为包含 `NCPU` 个槽位的数组，每个槽位拥有独立的 `kmem` 锁和空闲链表。所有锁名均以 `kmem` 开头，使课程统计系统能够汇总它们的竞争次数。

`kfree(pa)`

- `push_off()`，读取 `cpuid()`
- 获取 `kmem[id].lock`
- 页面插入当前 CPU `freelist` 表头
- 释放锁并 `pop_off()`

`kalloc()`

- `push_off()`，读取 `cpuid()`

- 从 `kmem[id].freelist` 取一页
- 本地为空时进入跨 CPU 偷取路径
- `pop_off()`, 返回页面

`cpuid()` 的结果只在中断关闭期间可靠；若中途发生调度并迁移到另一 CPU，继续访问旧编号对应的链表会破坏“常见操作保持本地”的性能目标。因此本实现从读取 CPU 编号到完成分配器操作一直保持 `push_off()`，内部普通自旋锁造成的嵌套关闭由 xv6 的 `noff` 计数正确配对。

9.2.3 设计与实现步骤

系统启动时初始化全部 `kmem[i].lock`，`freerange()` 仍按官方提示把初始空闲页交给执行初始化的 CPU。其他 CPU 第一次需要页面时才进行偷取。普通路径只持有一把本地锁，因此不同 CPU 上的高频单页分配和释放可以真正并行。

本地链表为空时，分配器依次检查其他 CPU。为了缩短跨 CPU 临界区，最终实现把来源链表整体转移到本地，并立即从表头取出一页，操作本身只需常数指针赋值。转移必须同时保护来源与目标链表；两把锁统一按 CPU 编号从小到大获取、反向释放，避免 CPU A 从 B 偷取、同时 B 从 A 偷取时形成循环等待。

本地 `freelist` 为空

- 选择 donor CPU
- 按 `min(id, donor)`、`max(id, donor)` 顺序获取两把锁
- donor 非空：整条链表转给本地，并取出第一页
- 反向释放两把锁
- donor 为空：继续检查下一 CPU

核心不变式是：每个空闲物理页始终位于某一条受锁保护的链表。整链表转移期间同时持有两端锁，所以其他 CPU 不会观察到页面既不属于来源、也不属于目标的状态。

9.2.4 实验结果

`./grade-lab-lock kallocetest` 的四项测试全部通过：test1 检查并发单页分配竞争，test2 检查低内存下反复统计不会丢页，test3 检查并发分配、回收与偷取的正确性，test4 检查大进程耗尽内存时的竞争性能。`usertests sbrkmuch` 也通过，证明所有物理内存仍可被完整分配。

```
== Test running kallocetest == (53.7s)
== Test    kallocetest: test1 ==
    kallocetest: test1: OK
== Test    kallocetest: test2 ==
    kallocetest: test2: OK
== Test    kallocetest: test3 ==
    kallocetest: test3: OK
== Test    kallocetest: test4 ==
    kallocetest: test4: OK
== Test kallocetest: sbrkmuch == kallocetest: sbrkmuch: OK (5.9s)
```

图 9-1 Memory allocator 五项专项测试全部通过

9.2.5 分析讨论

初版每次最多偷取 64 页，虽然 test1 至 test3 正确，但 test4 新增竞争约 54,715，超过 30,000 的评分阈值。把批次增至 1024 页又使来源锁内遍历时间过长，竞争进一步上升。这表明减少加锁次数不等于降低竞争，临界区长度同样重要。

随后尝试在来源锁内 O(1) 摘下整条链表、解锁后再挂入本地，但摘下到挂入之间，页面只存在于当前调用的局部变量中。其他 CPU 可能恰好检查完所有空链表并返回 内存不足，`kalloc_test` 因 `sbrk()` 返回 -1 后写地址 `0x3` 而失败。最终采用 固定顺序双锁下的 O(1) 转移，同时解决页面可见性、死锁与临界区长度三个问题，test4 的竞争计数降至评分阈值以内。

9.3 Read-write lock: 写者优先

9.3.1 状态设计与语义

普通自旋锁只允许一个持有者，即使多个临界区都只读取共享数据也会互相阻塞。读写锁需要满足：任意数量读者可以并发；写者与所有读者、其他写者互斥；一旦有写者开始等待，后来的读者不能继续插队。

`struct rwspinlock` 使用三个原子字段：`readers` 记录活动读者数，`writer` 表示 是否有活动写者，`waiting_writers` 记录已经宣布等待的写者数。所有状态使用 GCC `__atomic` 顺序一致操作，直接建立跨 CPU 的全序与内存屏障，避免普通读写的数据 竞争和弱内存序下的状态错判。

9.3.2 读者协议与竞态封闭

读者先等待 `writer == 0 && waiting_writers == 0`，再原子增加 `readers`。但首次 检查和增加计数之间可能恰好有写者宣布等待，所以增加后必须再次检查两个写者状态：若仍为零则成功获得读锁；否则撤销读者计数并重试。

```
等待 writer == 0 且 waiting_writers == 0
  → readers++
  → 再次检查写者状态
    └─ 仍无写者：获得读锁
    └─ 写者已出现：readers--，重新等待
```

二次检查保证两种合法顺序：读者若先完成登记，写者会等待它释放；写者若先宣布，读者就撤销并让路。不存在双方都认为自己已经获得锁的中间状态。

9.3.3 写者协议与优先级

写者首先增加 `waiting_writers`，从这一刻起后续读者都会停在入口。随后通过原子 `exchange` 竞争唯一的 `writer` 标志，成功后等待既有 `readers` 降为零，再减少 自己的等待计数并进入写临界区。释放时原子清除 `writer`。若还有其他写者等待，`waiting_writers` 仍大于零，读者继续等待，因此多个排队写者也不会被读者穿插。

公开的读写锁接口沿用框架提供的 `push_off()` / `pop_off()`，防止持锁代码被迁移，并允许通过嵌套中断计数在同一 CPU 上同时持有多把不同的读写锁。

9.3.4 专项验证

`rwlktest` 覆盖并发读、读写互斥、写写互斥、单写者优先、多写者优先和同时持有多把锁。专项测试连续运行四次均通过，四个 CPU 都返回 0：

```
lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$ ./grade-lab-lock rwlktest
tail -n 18 xv6.out
make: 'kernel/kernel' is up to date.
== Test running rwlktest == (5.8s)
== Test   rwlktest ==
    rwlktest: OK
rwspinlock_test: step 19: multiple writer priority test
rwspinlock_test: step 20: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 21: multiple writer priority test
rwspinlock_test: step 22: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 23: multiple writer priority test
rwspinlock_test: step 24: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 25: multiple writer priority test
rwspinlock_test: step 26: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 27: multiple writer priority test
rwspinlock_test: step 28: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 29: multiple writer priority test
rwspinlock_test: step 30: done
rwspinlock_test(0): 0
rwspinlock_test(1): 0
rwspinlock_test(2): 0
rwspinlock_test(3): 0
rwlktest: 4/4 CPUs succeeded
$ qemu-system-riscv64: terminating on signal 15 from pid 105992 (make)
lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$
```

图 9-2 Read-write lock 多写者优先测试与四 CPU 结果

9.4 实验中遇到的问题解决方法

本 Lab 不仅要求结果正确，还要求在多核压力下减少锁竞争并避免死锁。围绕锁粒度、所有权转移和写者优先进行了多轮调整。

原始物理页分配器只有一条空闲链表，所有 CPU 都要竞争 `kmem.Lock`。改为每 CPU 独立链表和锁后，常见分配路径不再跨核竞争，`kalloc_test test1` 通过。最初的偷页方案每次只移动少量页面，大内存申请仍会反复获取来源 CPU 的锁，后续改成 O(1) 整链表转移，`test4` 通过。

跨 CPU 转移页面时，若先从来源链表摘下页面并解锁，再挂入目标链表，这批页面在中间阶段对所有 CPU 都不可见。若两个 CPU 同时互相偷取且锁序相反，还会死锁。解决方法是让双锁覆盖完整所有权转移，并始终按 CPU 编号递增获取锁。高负载下 `test2`、`test3` 均通过且未发生死锁。

读写锁的第一版只检查是否有活动写者，读者可能在写者已经等待但尚未获得锁时继续插队。加入 `waiting_writers` 后，新读者在有等待写者时不再进入；读者登记后还会二次检查状态，发现竞争则撤销登记。单写者和多写者优先测试连续通过。

9.5 最终验收与心得

关键提交如下：

提交	内容
3bf1bcb	实现每 CPU 空闲链表、固定锁序和跨 CPU 整链表转移
d68c3cc	实现写者优先读写自旋锁
17aa0bc	记录 7 小时实际用时并完成验收

从干净状态运行 `make grade`，内存分配器四项、`sbrkmuch`、`rwlktest`、完整 `usertests` 和时间检查全部通过，最终得分 100/100：

```
== Test running kallocetest ==
$ make qemu-gdb
(53.5s)
== Test  kallocetest: test1 ==
kallocetest: test1: OK
== Test  kallocetest: test2 ==
kallocetest: test2: OK
== Test  kallocetest: test3 ==
kallocetest: test3: OK
== Test  kallocetest: test4 ==
kallocetest: test4: OK
== Test kallocetest: sbrkmuch ==
$ make qemu-gdb
kallocetest: sbrkmuch: OK (5.9s)
== Test running rwlktest ==
$ make qemu-gdb
(5.5s)
== Test  rwlktest ==
rwlktest: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (103.4s)
== Test time ==
time: OK
Score: 100/100
```

图 9-3 Lock Lab 完整评分结果 100/100

本 Lab 的核心认识是：锁的正确性只解决“能否运行”，锁的粒度、持有时间和获取顺序才决定多核程序“能否并行”。每 CPU 链表以空间分区减少共享，双锁协议维护跨分区转移不变式；读写锁则利用更细的访问语义扩大并发，同时用等待写者状态补上公平性。性能优化必须和状态可见性一起设计，不能以短暂破坏不变式换取更短的临界区。

10. Lab fs: File system

10.1 实验目的与要求

项目	内容
官方分支	fs
实验主题	inode 块映射、大文件存储与符号链接路径解析
具体任务	Large files、Symbolic links
基线提交	a95dec4
完成提交	1cec73d
实际用时	5 小时
最终评分	100/100

本 Lab 从 xv6 文件系统的两个互补方向展开。Large files 扩展单个 inode 能寻址的磁盘块数量，要求在磁盘 inode 大小不变的前提下加入二级间接索引；Symbolic links 则扩展命名语义，要求新增系统调用和 inode 类型，并在 `open()` 中安全地递归解析链接。两项任务都需要维护 inode、缓冲区、日志事务和引用计数的生命周期。

10.2 Large files: 二级间接块

10.2.1 实验目的

原始 xv6 inode 使用 12 个直接块地址和 1 个一级间接块地址。块大小为 1024 字节，每个块可保存 256 个 32 位块号，因此单文件最多只能包含 $12 + 256 = 268$ 个数据块。任务要求复用 inode 中一个直接地址槽作为二级间接根，在不增大磁盘 inode 的条件下支持 65,803 个数据块。

10.2.2 inode 布局与寻址原理

修改后的 13 个地址槽保持磁盘格式尺寸不变，但含义调整为：11 个直接块、1 个一级间接块和 1 个二级间接根块。

```
ip->addrs[0..10]  → 11 个直接数据块
ip->addrs[11]     → 一级间接块 → 256 个数据块
ip->addrs[12]     → 二级根块 → 256 个一级索引块
                  → 每个再指向 256 个数据块
```

对应常量为：

```
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
```

```
#define NDINDIRECT (NINDIRECT * NINDIRECT)
#define MAXFILE (NDIRECT + NINDIRECT + NDINDIRECT)
```

所以 $\text{MAXFILE} = 11 + 256 + 256 * 256 = 65803$ 。`struct dinode` 与内存中的 `struct inode` 均使用 `adrs[NDIRECT+2]`，总槽数仍为 13，避免改变 inode 在磁盘上的排列和每块 inode 数量。

10.2.3 设计与实现步骤

`bmap(ip, bn)` 先处理直接区和一级间接区。进入二级区域后，逻辑块号减去前两段容量，再分解为两级下标：

```
outer = bn / NINDIRECT
inner = bn % NINDIRECT
```

若二级根块不存在，则通过 `balloc()` 分配；随后读取根块，用 `outer` 找到或创建对应的一级索引块；最后读取该索引块，用 `inner` 找到或创建数据块。每次把新块号写入索引缓冲区后调用 `log_write()`，使元数据更新进入当前文件系统事务；每个 `bread()` 都在使用后执行 `brelse()`，避免泄漏缓冲区引用。

扩展寻址必须同步扩展删除路径。`itrunc()` 按从叶到根的顺序遍历二级树：先释放每个一级索引块指向的数据块，再释放该一级索引块，最后释放二级根块。若先释放索引块，就会丢失叶子块号并造成永久磁盘空间泄漏。完成释放后清空 inode 地址和大小，再由 `iupdate()` 持久化。

10.2.4 实验结果

专项命令 `./grade-lab-fs bigfile` 从新建文件开始连续写入 65,803 个块，再逐块读回校验。实际输出 `wrote 65803 blocks、bigfile done; ok`，评分通过：

```
== Test running bigfile (takes minutes) == running bigfile (takes minutes): OK (11
7.8s)
wrote 65803 blocks
reading bigfile
bigfile done; ok
```

图 10-1 Large files 写入并读回 65,803 个块

完整 `usertests` 同样通过，其中大文件写入、读取和删除路径验证了扩展后的 `bmap()` 与 `itrunc()` 能配合工作。

10.2.5 分析讨论

二级间接结构用额外一次磁盘索引访问换取约 256 倍的容量增长，同时仍保持小文件走直接块的快速路径。实现的关键不是下标计算本身，而是磁盘格式不变式和异常路径：磁盘与内存 inode 必须使用相同数量的地址槽；索引项修改必须写入日志；分配失败必须返回 0；释放必须由叶到根。只有映射和截断互为逆操作，文件反复创建、扩展、删除后磁盘空闲块计数才不会持续下降。

10.3 Symbolic links: 符号链接

10.3.1 系统调用与磁盘表示

新增 `symlink(target, path)` 系统调用后, 用户态声明、`usys.pl` 汇编桩、系统调用 编号和内核分派表形成完整调用链。文件类型增加 `T_SYMLINK`, 打开标志增加 `O_NOFOLLOW`。Makefile 同时把 `_symlinktest` 写入 fs 实验镜像。

`sys_symlink()` 在日志事务中调用 `create(path, T_SYMLINK, 0, 0)` 创建 inode, 再把 `target` 连同末尾 NUL 作为普通 inode 数据写入。创建过程不调用 `namei(target)`, 因此目标可以暂时不存在, 这符合符号链接区别于硬链接的语义。

```
symlink(target, path)
→ begin_op()
→ create(path, T_SYMLINK)
→ writei(ip, target, strlen(target) + 1)
→ iunlockput(ip)
→ end_op()
```

10.3.2 `open()` 的链接跟随

普通 `open()` 得到 inode 并加锁后, 若类型为 `T_SYMLINK` 且没有指定 `O_NOFOLLOW`, 就读取 inode 中保存的目标路径, 释放当前 inode 引用, 再对目标执行 `namei()` 和 `ilock()`。循环执行这一过程即可支持链接指向另一个链接。

实现对每层内容进行边界检查: 大小必须位于 `1..MAXPATH`, 读取长度必须完整, 最后一个字节必须为 NUL。解析深度限制为 10; 目标不存在、内容损坏或链接环导致深度超限时统一返回 `-1`, 并在错误路径释放 inode 锁和引用。设置 `O_NOFOLLOW` 时跳过该循环, 调用者得到链接 inode 本身。`link()`、`unlink()` 等操作没有加入跟随逻辑, 因此仍作用于目录项中的链接, 而不是它的最终目标。

10.3.3 并发与专项验证

链接创建和打开都位于 `begin_op()/end_op()` 事务范围内, inode 数据访问由 inode 锁串行化, 目录修改继续复用 `create()` 原有锁协议。因此并发进程创建、打开和删除不同链接时, 不需要另建全局链接表或锁。

`./grade-lab-fs symlinktest` 覆盖目标不存在、读写目标、`O_NOFOLLOW`、硬链接与 `unlink` 语义、链接链、循环检测和并发链接, 两项评分均通过:

```
lbyy@LAPTOP-US2J300J:~/xv6-labs-2025$ ./grade-lab-fs symlinktest
grep -E '^Start:|^test symlinks: ok$|^test concurrent symlinks: ok$' xv6.out
make: 'kernel/kernel' is up to date.
== Test running symlinktest == (1.8s)
== Test   symlinktest: symlinks ==
    symlinktest: symlinks: OK
== Test   symlinktest: concurrent symlinks ==
    symlinktest: concurrent symlinks: OK
Start: test symlinks
test symlinks: ok
Start: test concurrent symlinks
test concurrent symlinks: ok
```

图 10-2 Symbolic links 功能与并发测试全部通过

10.4 实验中遇到的问题和解决方法

文件系统修改必须同时保持磁盘格式、日志一致性、inode 锁和空间回收正确。开发中遇到的问题主要集中在索引层次和符号链接递归。

为 inode 增加二级间接地址时，如果直接扩充 `addrs` 数组，会改变磁盘 inode 尺寸，导致 `mkfs` 与内核使用不同的磁盘格式。实现将直接块数量从 12 减为 11，把空出的槽位用于二级间接块，使地址槽总数仍为 13，`mkfs` 和 `usertests` 均通过。

分配新的一级或二级索引块后，最初只修改了缓冲区内容，没有把指针更新加入日志，重启或异常时可能丢失映射。每次修改索引项后调用 `log_write()`，保证元数据进入当前事务。删除大文件时也必须从叶子数据块开始，依次释放一级索引块和二级根，否则会泄漏磁盘块。补全 `itrunc()` 后 `bigfile` 能完整读回，`usertests` 通过。

符号链接可以指向另一个符号链接，若存在环，`open()` 会无限解析。实现把最大跟随深度限制为 10 层，超限返回 `-1`。切换到下一目标前先对旧 inode 执行 `iunlockput()`，再 `namei()` 并锁定新 inode，避免引用泄漏和重复持锁。处理目录访问模式时还需屏蔽 `O_NOFOLLOW`，不能把它误当作读写模式。循环、并发和 `O_NOFOLLOW` 测试均通过。

10.5 最终验收与心得

关键提交如下：

提交	内容
707f919	实现二级间接块映射与完整截断释放
1e9024e	实现符号链接系统调用、递归跟随和循环限制
1cec73d	记录 5 小时实际用时并完成验收

从干净状态运行 `make grade`，Large files、Symbolic links 两项、完整 usertests 和 时间检查全部通过，最终得分为 100/100：

```
$ make qemu-gdb
running bigfile (takes minutes): OK (123.3s)
== Test running symlinktest ==
$ make qemu-gdb
(1.1s)
== Test   symlinktest: symlinks ==
   symlinktest: symlinks: OK
== Test   symlinktest: concurrent symlinks ==
   symlinktest: concurrent symlinks: OK
== Test usertests (takes minutes) ==
$ make qemu-gdb

usertests (takes minutes): OK (172.2s)
== Test time ==
time: OK
Score: 100/100
```

图 10-3 File System Lab 完整评分结果 100/100

本 Lab 展示了文件系统中“地址”和“名称”两层间接性。二级块索引树形结构把逻辑块号映射到磁盘块，符号链接则把路径名再次映射到另一个路径。两者都扩大了表达能力，也都引入了新的终止和回收问题：前者必须完整释放索引树，后者必须限制解析深度并正确交接 inode 引用。实现文件系统功能时，正常路径、日志持久性、锁、引用计数和失败回收必须作为同一套状态机一起设计。

11. Lab mmap: Memory mapping

11.1 实验目的与要求

项目	内容
官方分支	<code>mmap</code>
实验主题	文件映射、VMA、惰性缺页装入和共享写回
具体任务	实现 <code>mmap()</code> 、 <code>munmap()</code> 及 <code>fork/exit</code> 生命周期
基线提交	<code>5648f64</code>
完成提交	<code>9079100</code>
实际用时	8 小时
最终评分	170/170

本 Lab 在 xv6 中实现 Unix 文件内存映射的核心子集。`mmap()` 不立即分配物理页，只建立虚拟内存区域（VMA）；进程首次访问时由页错误处理程序读取对应文件页；`munmap()` 对共享映射写回修改并解除映射；`fork` 和 `exit` 则维护 VMA、物理页和文件引用的完整生命周期。实现把系统调用、页表、陷阱和文件系统四条路径连接起来。

11.2 系统调用与 VMA：只登记、不装页

11.2.1 实验目的

接入 `mmap(addr, len, prot, flags, fd, offset)` 和 `munmap(addr, len)` 两个系统调用，并为每个进程保存最多 16 个 VMA。课程限定 `addr=0`、`offset=0`，权限为读、写、执行位的组合，映射类型为 `MAP_PRIVATE` 或 `MAP_SHARED`。`mmap()` 必须快速返回，不能在调用时读取整个文件或分配所有物理页。

11.2.2 VMA 数据与地址分配

`struct proc` 中加入固定 16 项的 VMA 数组，每项记录：

字段	作用
<code>addr</code> 、 <code>length</code>	映射虚拟地址和原始字节长度
<code>prot</code> 、 <code>flags</code>	PTE 权限来源和共享/私有语义
<code>file</code>	被映射文件的独立引用
<code>offset</code>	VMA 起点对应的文件偏移

映射从 `0xc0000000` 起向上选择页对齐区间。分配器扫描所有有效 VMA；若候选区间重叠，就把候选地址推进到冲突 VMA 末尾并重新扫描，直到找到空洞或接近 `TRAPFRAME`。这种 first-fit 方法允许前缀/整段解除后复用空闲地址，而不依赖单调增长且可能溢出的全局指针。

`sys_mmap()` 验证长度、权限、flags、文件类型和可读性。共享可写映射还要求文件描述符可写；私有可写映射允许文件只读，因为修改只保留在私有物理页中。登记成功后调用 `filedup()`，所以用户随后 `close(fd)` 或 `unlink(path)` 都不会让 VMA 中的文件对象提前消失。

11.2.3 惰性设计的阶段验证

第一阶段只登记 VMA，尚未实现装页。`mmaptest` 的第一次 `mmap()` 已返回成功，程序在首次访问映射地址后产生 load page fault，并准确停在 `stval=0xc0000000`。加入缺页装入后，测试能够完整校验两页文件内容，继续推进到尚未实现的第一次 `munmap()`：

```
== Test running mmaptest == (1.1s)
test basic mmap
mmaptest failure: munmap (1), pid=3
$ qemu-system-riscv64: terminating on signal 15 from pid 150131 (make)
```

图 11-1 惰性装页完成后测试推进到 `munmap()` 阶段

这一步验证 `mmap()` 本身没有预读文件：若在系统调用中装页，后续测试修改文件后再访问映射时就无法观察到新内容，`lazy access` 会失败。

11.3 页错误：按文件偏移装入物理页

11.3.1 缺页分派与权限

`usertrap()` 将 instruction、load、store page fault (scause 12、13、15) 交给 `vmfault()`。处理程序先查找包含 fault VA 的 VMA，再根据访问类型检查 `PROT_EXEC`、`PROT_READ` 或 `PROT_WRITE`；不符合权限的访问返回失败，由 trap 路径终止进程。这样只读映射在第一次 store 时不会被错误地装成可写页。

命中合法 VMA 后，地址向下对齐到页边界，分配并清零一页物理内存。文件读取偏移为：

```
file_offset = vma.offset + (page_va - vma.addr)
```

处理程序持有 inode 睡眠锁调用 `readi()` 读取最多 4096 字节。页面预先清零，因此文件末尾不足一页的剩余空间自然为零。随后从 `prot` 构造 PTE：可写页同时设置 `PTE_R|PTE_W`，避免 RISC-V 保留的 `W=1,R=0` 编码；可执行页设置 `PTE_X`；所有映射加入 `PTE_U`。

内核复制路径也必须认识 VMA。`copyout()` 以 store 语义触发装页，`copyin()` 和 `copyinstr()` 以 load 语义装页，避免系统调用把映射区作为缓冲区时绕过权限或直接失败。未命中 VMA 时，`vmfault()` 继续执行原有 lazy sbrk 零页逻辑。

11.4 munmap：部分解除与共享写回

11.4.1 范围和 VMA 更新

课程允许解除 VMA 前缀、后缀或整段，但不要求在中间打洞。`vmaunmap()` 要求地址 页对齐、长度非零、取整后无溢出，并确认范围完全位于单个 VMA 且接触其中一个 端点。`uvmunmap()` 可以跳过不存在的 PTE，所以从未访问的惰性页无需先分配即可 直接解除。

解除整段时关闭 VMA 文件引用并清空槽位；解除前缀时同步推进 `addr` 和 `offset`，再缩短 `length`；解除后缀时保留起点，仅缩短长度。地址、文件偏移和长度必须一起 更新，否则后续页错误会从错误的文件位置读取。

11.4.2 MAP_SHARED 写回

对 `MAP_SHARED|PROT_WRITE` 的已装入页，解除前通过 PTE 取得物理地址，以内核源 地址调用 `writel()`。实现没有依赖 dirty 位，按课程允许的方式写回解除范围中的 所有已映射可写页；`MAP_PRIVATE` 不写回。

写回还处理了两个边界。第一，长度不超过 VMA 实际字节数和 inode 当前大小，避免 把 1.5 页文件因第二页尾部零填充错误扩展为 2 页。第二，按 `BSIZE` 分块，每块使用 独立的 `begin_op() / end_op()` 事务，避免一次写回 4 KiB 超过 xv6 的小型日志预算。

11.5 fork、exit 与资源生命周期

`kfork()` 复制父进程全部有效 VMA，并对每个文件执行 `filedup()`。课程允许父子 进程不共享同一物理页，因此高地址 VMA 页不由 `uvmcopy()` 复制；子进程首次访问时 根据相同 VMA 从文件重新装页。这样 `MAP_PRIVATE` 的父子修改天然隔离，减少了物理 页引用计数和 COW 的额外复杂度。

`kexit()` 对每个剩余 VMA 调用完整范围的 `vmaunmap()`，复用共享写回、页释放和 文件关闭逻辑。若极端写回错误，退出路径仍强制解除映射并释放引用，保证后续 `freewalk()` 不会遇到残留叶子 PTE。文件引用从 `mmap()` 的 `filedup()` 开始，最终只在整段 `munmap` 或 `exit` 时对应释放，形成闭合生命周期。

11.6 实验中遇到的问题和解决方法

`mmap` 把系统调用、页表、trap 和文件系统放入同一条延迟执行路径，因此很多错误 会在解除映射或进程退出时才暴露。

测试阶段曾把 `test basic mmap` 等 xv6 输出当作命令输入 Ubuntu Bash，因而出 现 `test: unary operator expected` 和 `command not found`。正确方式是在 Ubuntu 中运行 `./grade-lab-mmap mmaptest`，或者进入 QEMU 后只输入 `mmaptest`。区分两个 终端后，测试可以正常启动并输出逐项结果。

惰性装页完成后，进程退出曾触发 `panic: freewalk: leaf`。VMA 位于 `p->sz` 之外，普通地址空间释放路径不会遍历这些高地址叶子。解决方法是在 `freewalk()` 前逐个解除剩余 VMA，使共享写回、物理页释放和文件引用关闭都由统一路径完成。

构造 PTE 权限时，只设置 `PTE_W` 会形成 RISC-V 保留的 `W=1,R=0` 编码，页面会持续缺页，因此可写映射同时加入 `PTE_R|PTE_W`。缺页处理还要根据 `scause` 区分 `load` 和 `store`，否则只读映射可能被错误写入。`read/write` 和 `read-only write` 测试随后通过。

共享映射写回时，按整页写入会把文件尾页的零填充也写到 EOF 之后，使短文件被意外扩展；一次写回完整 4 KiB 还可能超过 xv6 日志容量。实现按 `inode` 当前大小截断末页，并按 `BSIZE` 拆成独立事务，`dirty` 和 `shared writeback` 测试通过。

部分解除 VMA 前缀时，最初只移动虚拟地址，没有同步推进文件偏移，后续缺页会从错误位置读取。修正为同时更新 `addr`、`offset` 和 `length` 后，`partial unmap` 通过。`fork` 时复制 VMA 并对文件执行 `filedup()`，子进程仍按需装页，`fork test` 也通过。

11.7 最终验收与心得

关键提交如下：

提交	内容
a6efd1e	接入 mmap/munmap 系统调用并建立 VMA 表
f65416c	实现文件映射页的惰性缺页装入和权限控制
fe7f9a7	实现部分 munmap 与 MAP_SHARED 分块写回
dd62bce	实现 fork VMA 继承和 exit 自动解除
9079100	记录 8 小时实际用时并完成验收

从干净状态运行 `make grade`，11 项 `mmaptest`、完整 `usertests` 和时间检查全部通过，最终得分 170/170：

```
== Test running mmaptest ==
$ make qemu-gdb
(3.0s)
== Test  mmaptest: mmap basic ==
mmaptest: mmap basic: OK
== Test  mmaptest: mmap private ==
mmaptest: mmap private: OK
== Test  mmaptest: mmap read-only ==
mmaptest: mmap read-only: OK
== Test  mmaptest: mmap read/write ==
mmaptest: mmap read/write: OK
== Test  mmaptest: mmap dirty ==
mmaptest: mmap dirty: OK
== Test  mmaptest: not-mapped unmap ==
mmaptest: not-mapped unmap: OK
== Test  mmaptest: lazy access ==
mmaptest: lazy access: OK
== Test  mmaptest: two files ==
mmaptest: two files: OK
== Test  mmaptest: fork_test ==
mmaptest: fork_test: OK
== Test  mmaptest: munmap_noaccess ==
mmaptest: munmap_noaccess: OK
== Test  mmaptest: read_only_write ==
mmaptest: read_only_write: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (40.4s)
== Test time ==
time: OK
Score: 170/170
```

图 11-2 mmap Lab 完整评分结果 170/170

本 Lab 的核心认识是：内存映射不是单次系统调用，而是跨越整个进程生命周期的延迟协议。VMA 描述承诺，页错误兑现页面，`munmap` 和 `exit` 回收资源，`fork` 复制 承诺但不必复制已兑现的物理页。任何一个环节遗漏权限、偏移、引用或写回，都会在很晚的执行阶段才表现为数据损坏或页表 panic，因此必须用统一的不变式分析全链路。

12. 综合分析

12.1 系统调用的完整链路

util Lab 从用户程序一侧调用 `pause`、`open`、`read`、`fork` 和 `exec`；syscall Lab 则补齐了另一侧的入口和分派过程。完整链路为：用户 C 函数调用由 `usys.pl` 生成的汇编桩，桩函数把调用号写入 `a7` 后执行 `ecall`；硬件切换 特权级，trampoline 保存寄存器，`usertrap()` 进入 `syscall()`；分派器根据 `trapframe->a7` 查表调用 `sys_*` 实现，并把返回值写回 `a0`。新增 `interpose()` 证明扩展系统调用必须同时修改声明、桩、编号、分派和内核实现，而 sandbox 还说明系统调用策略属于进程状态，必须随 `fork` 正确继承。fs Lab 的 `symlink()` 再次经过同一条接入链路，并进一步进入日志事务、目录创建和 `inode` 写入，展示系统调用如何把用户路径参数转化为持久化文件系统状态。

12.2 进程、陷阱与虚拟内存的关系

syscall、pgtbl、traps 和 COW Lab 已展示三者的基本关系：进程通过 `struct proc` 持有独立页表和 `trapframe`；用户态执行 `ecall` 或发生页错误时，硬件进入 supervisor mode，trampoline 利用高地址处的 `trapframe` 映射保存上下文；内核处理完成后恢复用户页表和寄存器。`USYSCALL` 表明同一进程页表可同时包含普通用户页、用户只读的内核共享数据以及仅 supervisor 可访问的 `trapframe` 与 trampoline。COW 进一步利用只读 PTE 主动制造写缺页，在 `vmfault()` 中把共享物理页拆分为私有副本，说明页错误既可能代表非法访问，也可以成为延迟执行内存操作的正常控制流。mmap 进一步把这种机制推广到文件映射：VMA 先记录地址范围、文件和权限，首次访问再由同一缺页入口分配物理页并从文件读取。COW 延迟“复制”，mmap 延迟“读取”，两者都用 PTE 状态和 trap 把原本集中执行的内存操作分散到实际访问时完成；区别在于 COW 页的后端是已有物理页，mmap 页的后端是 `inode` 数据，并且共享映射还必须在解除映射或退出时完成文件写回。

12.3 并发、锁与资源生命周期

Network Lab 同时展示了两类生产者—消费者同步。E1000 描述符环由 CPU 与设备 并发访问，软件根据 `DD` 判断设备是否完成，并用内存屏障保证描述符内容先于 tail 更新可见；UDP 端口队列则由中断接收路径生产、用户进程消费，通过 `netlock` 和 `sleep/wakeup` 保证队列不变式与阻塞唤醒不丢失。两条路径共同遵循 缓冲页所有权规则：每个时刻只能有一个组件负责最终释放页面，成功移交后原持有者不再释放，移交失败则立即回收。Lock Lab 进一步说明，共享状态正确并不意味着 并行度足够：把物理页按 CPU 分区后，常见分配路径不再争用全局锁；跨分区转移则依靠固定双锁顺序同时保证可见性和无死锁。读写锁又把访问区分为读与写，通过 活动读者、活动写者和等待写者三个状态，在扩大读并行度的同时保证写者优先。fs Lab 中 `bmap()` 和 `itrunc()` 则依靠 `inode` 锁保护单个文件的块映射，并通过 `bread()/brelse()` 管理缓冲区引用、`log_write()` 把元数

据变更纳入事务。它与前述内存并发路径的共同点是：锁只保护内存中的瞬时状态，资源所有权和持久化顺序还需要独立的不变式约束。

12.4 各 Lab 之间的联系

全部九个 Lab 构成一条逐层深入的路径：util 在用户态组合系统调用实现命令；syscall 进入内核观察调用分派、策略控制和隔离漏洞；pgtbl 继续下降到支撑进程隔离的地址转换和物理页生命周期。attack 能泄露秘密的根因正是物理页重用时没有清零，而 pgtbl Lab 的 kalloc、uvmmalloc、uvmcopy 和 uvmunmap 进一步展示了这些页面如何分配、映射、复制和回收。traps 则解释了用户程序如何经 ecall 和 trapframe 穿过特权级边界，并通过 Backtrace 与 Alarm 分别观察同步系统调用链和异步时钟中断下的控制流保存与恢复。COW 在这些基础上把页表权限、页错误和物理页生命周期组合成延迟复制机制，展示了虚拟内存如何用一次额外的间接层换取更低的 fork 时间和内存开销。net Lab 则把系统边界从用户/内核继续延伸到外部设备：中断机制接收网卡事件，页分配器提供 DMA 缓冲区，锁和睡眠唤醒连接异步中断与阻塞系统调用，copyout() 最终把 UDP 数据交还用户地址空间。此前各 Lab 中分别学习的陷阱、内存、并发和系统调用机制，因此在一条完整的数据包接收链上组合起来。lock Lab 回到这些公共内核机制的并发基础，直接优化 COW、页表、网络缓冲区等路径共同依赖的物理页分配器，并通过读写锁展示“访问语义比互斥更细”时可获得的并行性。它把前面实验中作为工具使用的锁，变成了需要证明不变式、分析交错顺序和测量竞争开销的研究对象。fs Lab 最后把系统调用、锁和资源生命周期落实到持久化存储：二级间接块扩展 inode 的数据寻址能力，符号链接扩展路径名称空间；日志保证索引更新的事务性，按层释放与解析深度限制分别保证空间和控制流终止。mmap Lab 在此基础上把文件内容直接接入用户虚拟地址空间。VMA 将文件引用和地址范围绑定，页错误按需建立文件页，munmap() 与 exit 负责共享写回和最终回收，因此前面分别实现的系统调用、页表、trap、锁和 inode 操作在同一生命周期中汇合。

13. 总结与心得

本项目完成了 MIT 6.1810 2025 版九个 xv6 Labs，并通过各分支的完整评分。实验过程使我对操作系统的理解从“各模块提供什么功能”转向“状态如何跨模块流动”。一次用户请求可能依次经过系统调用桩、trapframe、内核分派、页表、锁和文件系统；任何一层遗漏权限检查、引用释放或失败回滚，问题都可能在很晚的位置才出现。

实现中最重要的方法是先确定不变式，再编写正常路径。例如 COW 要保证每个共享页的引用计数与映射数一致；网卡收发要保证描述符和缓冲页只有一个释放责任方；文件系统要保证索引更新进入日志且截断能完整回收；mmap 则要保证 VMA、PTE、文件引用和写回范围始终对应。围绕这些不变式进行分阶段测试，比只根据最终输出修补代码更容易定位根因，也能降低后续功能引入回归的概率。

九个 Lab 还展示了性能优化与正确性的关系。超级页、COW、每 CPU 空闲页链表和读写锁都通过减少复制、映射或锁竞争提升性能，但同时引入了降级、引用计数、偷取顺序和公平性等额外状态。后续可继续使用 QEMU 多核压力测试、故障注入和性能计数扩大验证范围，并把当前手工整理的阶段证据进一步自动化。总体而言，本项目建立了从用户态接口到硬件边界、从瞬时内存状态到持久化数据的完整 xv6 实践认识。

14. 参考资料

1. MIT 6.1810 Fall 2025 Course Website: <https://pdos.csail.mit.edu/6.828/2025/>
2. MIT 6.1810 Tools: <https://pdos.csail.mit.edu/6.828/2025/tools.html>
3. MIT 6.1810 Lab Guidance: <https://pdos.csail.mit.edu/6.828/2025/labs/guidance.html>
4. MIT 6.1810 Lab: Xv6 and Unix utilities: <https://pdos.csail.mit.edu/6.828/2025/labs/util.html>
5. MIT 6.1810 Lab: System calls: <https://pdos.csail.mit.edu/6.828/2025/labs/syscall.html>
6. MIT 6.1810 Lab: Page tables: <https://pdos.csail.mit.edu/6.828/2025/labs/pgtbl.html>
7. MIT 6.1810 Lab: Traps: <https://pdos.csail.mit.edu/6.828/2025/labs/traps.html>
8. MIT 6.1810 Lab: Copy-on-Write Fork: <https://pdos.csail.mit.edu/6.828/2025/labs/cow.html>
9. MIT 6.1810 Lab: Network Driver: <https://pdos.csail.mit.edu/6.828/2025/labs/net.html>
10. MIT 6.1810 Lab: Locks: <https://pdos.csail.mit.edu/6.828/2025/labs/lock.html>
11. MIT 6.1810 Lab: File System: <https://pdos.csail.mit.edu/6.828/2025/labs/fs.html>
12. MIT 6.1810 Lab: Mmap: <https://pdos.csail.mit.edu/6.828/2025/labs/mmap.html>
13. Russ Cox, Frans Kaashoek, Robert Morris. xv6: a simple, Unix-like teaching operating system.
14. RISC-V International. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture.

15. 源码仓库与提交记录

- 源码仓库: <https://github.com/lbyy718/xv6-labs-2025>
- 官方基线远程: `mit`
- 个人仓库远程: `origin`
- 各 Lab 使用同名分支保存实现。

Lab	分支	关键提交					最终评分
util	<code>util</code>	6bff0db	10d4cfd	52d4487	c28178d	5493306	131/131
syscall	<code>syscall</code>	1c298e3	ee2117e	6cfe037	346d204	f031d96	45/45
pgtbl	<code>pgtbl</code>	6097a95	781903e	d8c29ac	b7b1643	364eff0	41/41
traps	<code>traps</code>	c572d63	59b7b78	5abddb9	895895c		95/95
cow	<code>cow</code>	ecafd04	7fc6398				130/130
net	<code>net</code>	3edc8d7	8096c80	d9ad0dc			171/171
lock	<code>lock</code>	3bf1bcb	d68c3cc	17aa0bc			100/100
fs	<code>fs</code>	707f919	1e9024e	1cec73d			100/100
mmap	<code>mmap</code>	a6efd1e	f65416c	fe7f9a7	dd62bce	9079100	170/170

附录：完整评分结果

Lab	命令	结果	证据
util	make clean && make grade	131/131	图 3-4
syscall	make grade	45/45	图 4-4
pgtbl	make grade	41/41	图 5-4
traps	make grade	95/95	图 6-4
cow	make grade	130/130	图 7-1
net	make grade	171/171	图 8-3
lock	make grade	100/100	图 9-3
fs	make grade	100/100	图 10-3
mmap	make grade	170/170	图 11-2